MMSkills: Towards Multimodal Skills for General Visual Agents

Kangning Zhang[1,2‡*], **Shuai Shao**[1,2‡*], **Qingyao Li**[1,2*], **Jianghao Lin**[1], **Lingyue Fu**[1], **Shijian Wang**[3], **Wenxiang Jiao**[2,✉], **Yuan Lu**[2,✉], **Weiwen Liu**[1,✉], **Weinan Zhang**[1,✉], **Yong Yu**[1,✉]

[1]Shanghai Jiao TongUniversity, [2]Xiaohongshu Inc., [3]Southeast University

*Work done during internship at Xiaohongshu Inc., ‡Equal contribution, ✉Corresponding authors

Reusable skills have become a core substrate for improving agent capabilities, yet most existing skill packages encode reusable behavior primarily as textual prompts, executable code, or learned routines. For visual agents, however, procedural knowledge is inherently multimodal: reuse depends not only on what operation to perform, but also on recognizing the relevant state, interpreting visual evidence of progress or failure, and deciding what to do next. We formalize this requirement as *multimodal procedural knowledge* and address three practical challenges: (I) **what** a multimodal skill package should contain; (II) **where** such packages can be derived from public interaction experience; and (III) **how** agents can consult multimodal evidence at inference time without excessive image context or over-anchoring to reference screenshots. We introduce *MMSkills*, a framework for representing, generating, and using reusable multimodal procedures for runtime visual decision making. Each MMSkill is a compact, state-conditioned package that couples a textual procedure with runtime state cards and multi-view keyframes. To construct these packages, we develop an agentic trajectory-to-skill Generator that transforms public non-evaluation trajectories into reusable multimodal skills through workflow grouping, procedure induction, visual grounding, and meta-skill-guided auditing. To use them, we introduce a branch-loaded multimodal skill agent: selected state cards and keyframes are inspected in a temporary branch, aligned with the live environment, and distilled into structured guidance for the main agent. Experiments across GUI and game-based visual-agent benchmarks show that MMSkills consistently improve both frontier and smaller multimodal agents, suggesting that external multimodal procedural knowledge complements model-internal priors.

Contact: zhangkangning@sjtu.edu.cn, wenxiangjiaonju@gmail.com, liuww@sjtu.edu.cn
Code & Demo: https://github.com/DeepExperience/MMSkills
Skill Source: huggingface.co/datasets/zhangkangning/mmskills
Website: zkangning.github.io/towards_mmskills

1 Introduction

Skills have become one of the central abstractions for building useful agents: recent systems store reusable behaviors as prompts, code, execution graphs, or learned routines that can be retrieved and composed later (Wang et al., 2023a; Zheng et al., 2025; Chen et al., 2026; Wang et al., 2026a). Despite differences in implementation, these skills largely share a common representational assumption: reusable knowledge can be expressed as a textual or code-level specification of actions. This design is effective when the relevant state can be adequately abstracted in language, but it is insufficient for multimodal agents whose decisions depend on visual evidence. For such agents, reusable experience must specify not only what operation to perform, but also how to recognize the relevant state, and how visual evidence should guide the next decision. A desktop agent may know the correct operation but fail to recognize that a dialog is not yet ready; a game agent may know the intended goal but still require visual cues to distinguish progress from completion. This observation is consistent with human procedural learning, where visual information can complement verbal explanations (Mayer, 2009). Consequently, text-only skills become verbose yet underspecified, whereas demonstrations preserve visual context but are lengthy, instance-specific, and difficult to adapt.

This gap suggests the need for *multimodal procedural knowledge*: reusable guidance that binds action procedures to the visual evidence and state-dependent decisions required for applying them. Such knowledge is not simply a text skill with screenshots attached. To be reusable, it must specify what procedure is being reused, when the procedure

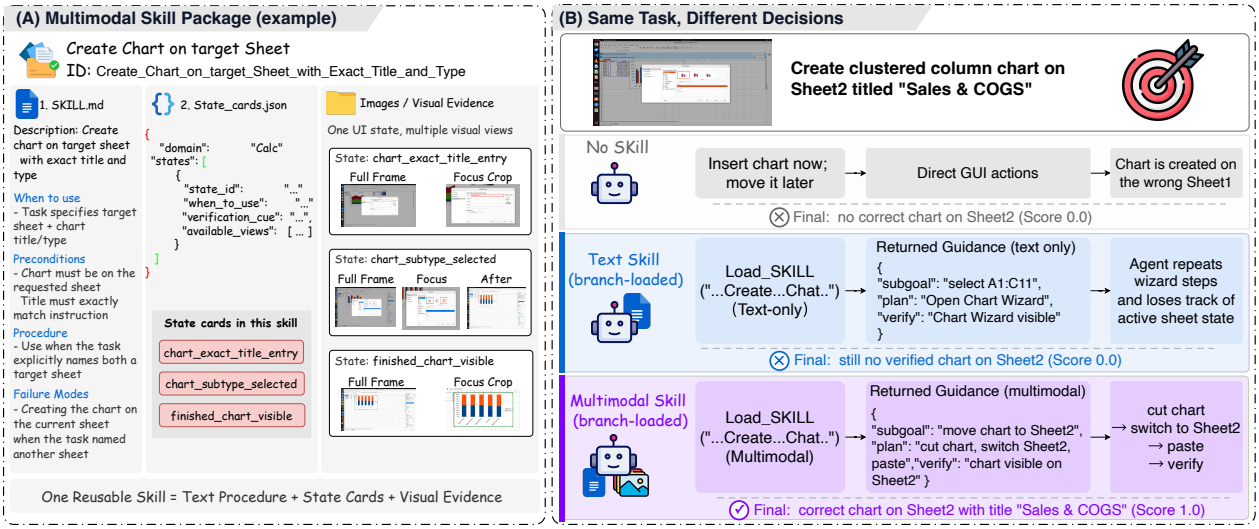

Figure 1 A concrete MMSkills example. A multimodal skill package combines a textual procedure, runtime state cards, and multi-view visual evidence. For the same chart-creation task, text-only guidance can miss the active sheet state, while branch-loaded MMSkills align skill evidence with the live screen and return state-aware guidance for the main agent.

should or should not be used, which visible cues matter, and which evidence verifies progress, failure, or completion. Turning this requirement into practical multimodal skill libraries raises three central challenges:

- **Representation.** What should a multimodal skill package contain, and how should it bind procedures, visible, and verification cues into a coherent reusable unit?
- **Generation.** Where can such packages be derived from, if they must use public non-evaluation interaction experience rather than hand-written examples or raw demonstration replay?
- **Utilization.** How can an agent consult multimodal skill evidence at inference time while avoiding excessive image context, distracting state descriptions, and over-anchoring to reference screenshots?

We propose **MMSkills**, a framework for representing, generating, and utilizing reusable multimodal procedures for runtime visual decision making. Each MMSkill couples a *textual procedure*, which describes the reusable action pattern, with *runtime state cards*, which encode when-to-use and when-not-to-use conditions, visible cues, verification cues, and available views, and *multi-view keyframes*, which ground critical states through full-frame, focused, and optional before/after views. The resulting package is not a text instruction with illustrative images attached. It is a state-conditioned procedure whose visual evidence helps the agent decide when to follow, skip, or verify the procedure.

To **generate** the multimodal skill package, we introduce an **automated trajectory-to-skill Generator** built around an agentic, meta-skill-guided pipeline. This generation problem is substantially harder than text-skill extraction: while prior pipelines can often compress successful rollouts, failure analyses, or accumulated traces into reusable instructions or action abstractions (Zheng et al., 2025; Wang et al., 2026a; Alzubi et al., 2026; Ma et al., 2026; Xia et al., 2026; Li et al., 2026b), generating MMSkills must also identify reusable visual states, select diagnostic frames, and bind each visual cue to the decision rule it supports. Our Generator operates on public trajectories that are **separate from evaluation tasks**: it groups related workflows, induces candidate procedures, merges overlapping candidates, grounds them in real non-test trajectory frames, and audits the resulting packages with reusable multimodal-skill-factory meta-skills. This process converts public interaction data into compact visual procedural knowledge without storing raw demonstrations as the skill.

For effective **utilization**, we introduce **branch loading** to consult the multimodal skills without injecting the entire package into the main trajectory. Existing skill agents commonly insert retrieved skills directly into the main interaction context. This loading pattern becomes problematic for MMSkills: a single package may contain several state cards together with multi-view screenshots, so direct insertion creates substantial context pressure and makes reference images compete with the live observation. More importantly, the main agent can become visually anchored to superficially similar reference screenshots, planning around the skill example rather than the current environment. Branch loading addresses this issue as a multimodal form of progressive disclosure over skill evidence (Xu and Yan,

2026). When the main agent considers a skill, it opens a temporary branch that selects the needed state cards and keyframe views, aligns them with the live screen or scene, and returns compact structured guidance with applicability judgments, subgoals, and next-step plans. The main trajectory receives distilled decision support rather than the full skill package, as illustrated by the example in Figure 1.

We evaluate MMSkills across GUI and game-based visual agent tasks, including OSWorld (Xie et al., 2024), macOSWorld (Yang et al., 2025b), VAB-Minecraft from VisualAgentBench (Liu et al., 2024a), and Super-Mario in LMGame-Bench (Hu et al., 2025). Across frontier and smaller multimodal models, MMSkills improve performance over no-skill and text-only skill conditions, suggesting that external visual procedural knowledge complements model-internal priors.

Our main contributions are summarized as follows:

- To the best of our knowledge, we are the first to introduce the **multimodal skill package**, formulating reusable skills for general visual agents as multimodal procedural knowledge: compact, state-conditioned units that organize textual procedures, runtime state cards, and multi-view keyframes for visual decision making.
- We develop an agentic trajectory-to-skill **Generator** that turns public, non-evaluation trajectories into multimodal skill packages through workflow grouping, procedure induction, visual grounding, and meta-skill-guided auditing.
- We propose **branch loading**, a runtime mechanism that selects and aligns multimodal skill evidence in a temporary branch before returning structured decision support to the main agent.
- We demonstrate significant gains across GUI and game-based visual-agent benchmarks and multiple model families, showing that external multimodal procedural knowledge complements model-internal priors.

2 Methods

2.1 Overview

MMSkills are designed around three components: a *multimodal skill package* that stores reusable visual procedural knowledge, a *Skill Generation pipeline* that constructs such packages from public trajectories, and a *branch-loaded multimodal skill agent* that isolates skill-environment grounding in a temporary branch and returns distilled decision support to the main trajectory at inference time. Figure 2 gives the system overview.

At a high level, the Generator maps non-evaluation trajectories $\mathcal{T} = \{\tau_i\}$ into a multimodal skill library $\mathcal{M} = \{M_i\}_{i=1}^{N}$. Before an episode begins, the runtime agent pre-recalls a task-level candidate set $\mathcal{C}_I \subset \mathcal{M}$ from the instruction I and compact skill descriptors. During execution, the main agent observes the current visual observation O_t, maintains a short history H_t, and either acts directly or consults a temporary skill branch for some $M_t \in \mathcal{C}_I$:

$$\begin{aligned} \text{direct}: &\quad A_t = \pi_{\text{main}}(O_t, H_t, \mathcal{C}_I), \\ \text{branch}: &\quad G_t = \text{Branch}(O_t, H_t, M_t), \quad A_t = \pi_{\text{main}}(O_t, H_t, \mathcal{C}_I, G_t). \end{aligned} \tag{1}$$

The branch output is a structured guidance tuple

$$G_t = (\text{applicable}_t, \text{subgoal}_t, \text{plan}_t, \text{do\_not\_do}_t, \text{verify}_t), \tag{2}$$

where the fields respectively give the applicability judgment, local subgoal, skill-conditioned plan, negative constraints, and visual verification check. The main agent uses G_t as decision support, while executable action grounding remains tied to the live observation.

2.2 Multimodal Skill Package

We represent each MMSkill as a state-conditioned procedure package

$$M = (D, P, S, K), \tag{3}$$

where D is a compact descriptor, P is a reusable textual procedure, $S = \{S_j\}_{j=1}^{m}$ is a set of runtime state cards, and $K = \{K_j\}_{j=1}^{m}$ is a set of keyframe bundles aligned with those cards. Each pair (S_j, K_j) corresponds to one

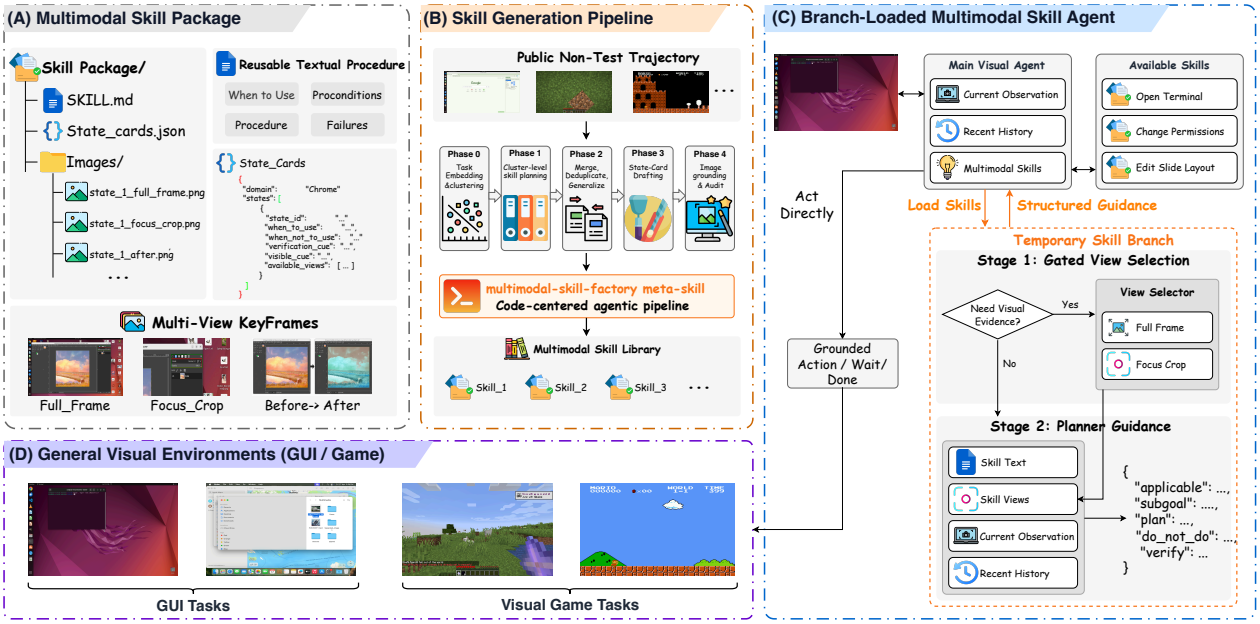

Figure 2 Overview of the MMSkills framework. A multimodal skill package stores a reusable textual procedure, runtime state cards, and multi-view keyframes. A meta-skill-guided Generator converts public non-test trajectories into a reusable multimodal skill library. At inference time, the main visual agent uses branch loading to inspect selected skill evidence in a temporary branch and receives compact structured guidance before acting.

decision-relevant procedural state. The procedure specifies the reusable workflow; the state card specifies when the workflow is valid or invalid; and the keyframes make the state visually recognizable at runtime.

A runtime state card is an agent-facing state node rather than an image caption. It links a point in the procedure to when-to-use conditions, when-not-to-use conditions, visible cues, verification cues, and available views:

$$S_j = (\text{when\_to\_use}_j, \text{when\_not\_to\_use}_j, \text{visible\_cues}_j, \\ \text{verification\_cue}_j, \mathcal{V}_j), \qquad \mathcal{V}_j = \text{available\_views}_j. \tag{4}$$

The first two fields define when the state should be followed or skipped, $\texttt{visible\_cues}_j$ states what evidence to inspect, $\texttt{verification\_cue}_j$ defines the progress or completion check, and $\mathcal{V}_j$ lists which views may be loaded. This schema makes the skill useful for decision making: the agent can decide whether to follow, skip, or verify the procedure.

Each key state is grounded by a small multi-view bundle. Let

$$\mathcal{V} = \{\text{full\_frame}, \text{focus\_crop}, \text{before}, \text{after}\}. \tag{5}$$

Then

$$K_j = \{K_j^v : v \in \mathcal{V}_j,\ v \in \mathcal{V}\}. \tag{6}$$

The full-frame view preserves global context, the focus crop localizes the visual cue, and optional before/after views expose useful transitions. These images are reference evidence, not coordinates to copy. Under this representation, a text-only skill is the degenerate package $(D, P, \emptyset, \emptyset)$; MMSkills extend it by binding procedure, decision conditions, and visual evidence into one reusable unit.

2.3 Skill Generator from Public Trajectories

We build MMSkills from public interaction trajectories that are separate from evaluation tasks. A trajectory is

$$\tau_i = (I_i, O_{i,1:T_i}, A_{i,1:T_i}), \tag{7}$$

where I_i is the task instruction, $O_{i,t}$ are visual observations, $A_{i,t}$ are executed actions. The Generator is controlled by a reusable multimodal-skill-factory meta-skill $\mathcal{F}$:

$$\mathcal{G}_{\mathcal{F}} : \mathcal{T}_d \mapsto \mathcal{M}_d, \tag{8}$$

where $\mathcal{T}_d$ is the public trajectory pool for domain d and $\mathcal{M}_d$ is the generated domain skill library. The pipeline comprises five stages:

$$\mathcal{T}_d \xrightarrow{\text{Phase 0: embed+cluster}} \mathcal{C}_d \xrightarrow{\text{Phase 1: cluster plan}} \mathcal{A}_d \xrightarrow{\text{Phase 2: merge}} \mathcal{R}_d \xrightarrow{\text{Phase 3: text draft}} \widehat{\mathcal{M}}_d \xrightarrow{\text{Phase 4: image ground+audit}} \mathcal{M}_d. \tag{9}$$

- **Phase 0: task embedding and clustering.** The pipeline embeds task instructions and trajectory metadata, then groups a broad domain into semantically focused clusters $\mathcal{C}_d$.
- **Phase 1: cluster-level skill planning.** For each cluster, an LLM-based agent proposes atomic skills with workflow boundaries, completion conditions, and covered task ids, producing a domain planning table $\mathcal{A}_d$.
- **Phase 2: skill merging.** Cluster-level plans are deduplicated, merged, and generalized into merged skill specifications $\mathcal{R}_d$, while overly broad umbrella skills are rejected.
- **Phase 3: text-first drafting.** Without reading images, the Generator selects reference tasks and drafts the descriptor D, textual procedure P, and planned state cards, yielding $\widehat{\mathcal{M}}_d$.
- **Phase 4: image grounding and audit.** The Generator reads selected keyframes, grounds focus regions, constructs multi-view bundles, and audits the final packages.

For a merged skill $r \in \mathcal{R}_d$, finalization is written as

$$\widehat{M}_r = (D_r, P_r, \widehat{S}_r, \widehat{K}_r) \xrightarrow{\text{ground+audit}} M_r = (D_r, P_r, S_r, K_r). \tag{10}$$

The visual grounding policy is conservative: views are added only for state recognition, transition comparison, or completion verification, so the skill stores diagnostic states rather than replaying demonstrations. The meta-skill $\mathcal{F}$ supplies reusable scripts, schemas, and quality gates for the LLM-based Generator, while external services are limited to bounded support steps such as embedding/clustering and grounding.

2.4 Branch-loaded Multimodal Skills Agent

Most skill-using agents load a retrieved skill directly into the main interaction context. For short text skills, this is reasonable: the skill is read as an additional instruction alongside the observation. For MMSkills, direct loading is brittle because state cards, multi-view keyframes, and transition examples add substantial context pressure, and irrelevant reference views can anchor the agent away from the live environment. Figure 2(C) illustrates the branch-loaded alternative, which moves skill-environment grounding out of the main trajectory.

Stage 1: gated view selection. Suppose the main agent calls $M_t = (D_t, P_t, S_t, K_t) \in \mathcal{C}_I$. The branch first selects which state cards and view types are relevant to the live observation:

$$(J_t, R_t) = \text{SelectViews}(O_t, H_{t-1}, P_t, S_t), \qquad V_t = \{K_j^v : j \in J_t,\ v \in R_{t,j}\}, \tag{11}$$

where J_t indexes selected state cards and $R_{t,j} \subseteq \mathcal{V}_j$ selects views for state j. The selector reads the live observation, recent history, textual procedure, and state-card descriptions before loading images. If text and state cards are sufficient, $R_{t,j}$ may be empty.

Stage 2: branch planning. The branch then aligns the selected evidence with the live state and returns structured guidance:

$$G_t = \text{PlanBranch}(O_t, H_{t-1}, P_t, \{S_j : j \in J_t\}, V_t), \tag{12}$$

where G_t follows Eq. 2. The main agent does not execute G_t mechanically; it uses G_t as an intermediate planning signal and still chooses a grounded action from the live screenshot. This preserves procedural guidance without allowing reference images to override the current observation. Appendix D gives the full runtime loop in Algorithm 1, and Appendix E reports the prompt templates used by the main agent and the two branch stages.

3 Experiments

We evaluate whether MMSkills provide useful external procedural knowledge for visual agents. The experiments are organized around four research questions:

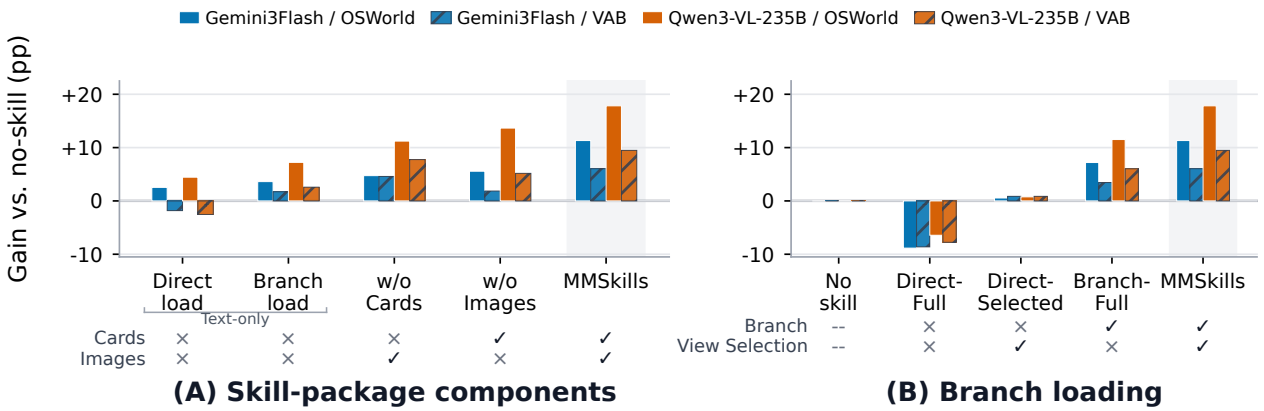

Figure 3 Ablation results for MMSkills components and branch loading. Bars report percentage-point gains over the no-skill baseline. Panel (A) removes runtime state cards or visual keyframes from the skill package. Panel (B) compares direct loading with branch loading and with or without view selection.

- **RQ1: Overall performance on GUI and game tasks.** Do MMSkills improve visual agents across realistic desktop environments and open-ended visual game tasks?
- **RQ2: Ablations of skill content and branch loading.** Which parts of MMSkills matter, and how do branch loading and view selection affect multimodal skill use?
- **RQ3: Skill usage and interaction dynamics.** How often are MMSkills invoked, how do they affect interaction length, and which visual views are selected at runtime?
- **RQ4: Behavioral shift analysis.** How do MMSkills change the agent's low-level action patterns beyond final success rate?

3.1 Experimental Setup

In all settings, agents plan from visual observations, namely desktop or game screenshots. We evaluate on OSWorld (Xie et al., 2024), macOSWorld (Yang et al., 2025b), VAB-Minecraft from VisualAgentBench (Liu et al., 2024a), and Super Mario Bros from LMGame-Bench (Hu et al., 2025), covering both realistic GUI tasks and open visual game environments. Detailed benchmark descriptions and test-case distributions are illustrated in Appendix A; implementation details, evaluation protocols, model choices, and runtime variants are given in Appendix C.

All skills are extracted from non-test data. We evaluate frontier and smaller multimodal models and compare *no-skill*, *text-only skill*, and *MMSkills* conditions, with direct-loading variants studied in the ablations. Dataset-specific skill sources, source statistics, and skill-package distributions are provided in Appendix B.

3.2 RQ1: Overall Performance on GUI and Game Tasks

Table 1 reports OSWorld application-level success rates, and Table 2 reports the auxiliary GUI and game results. **MMSkills improve OSWorld overall performance across all evaluated model families.** Overall success increases for Gemini 3.1 Pro (44.08% → 50.11%), Gemini 3 Flash (36.65% → 47.97%), Qwen3-VL-235B (21.34% → 39.17%), GLM-5V, and Kimi-K2.6. Text-only skills help but are less stable across domains, suggesting that procedures alone are insufficient when skill use depends on visual state matching. **External multimodal procedural knowledge is especially valuable for weaker visual agents.** For Qwen3-VL-8B-Instruct, MMSkills raise OSWorld from 10.78% to 25.40% and VAB-Minecraft from 23.28% to 38.79%, indicating that explicit visual procedural knowledge can compensate for limited model-internal priors.

The gains transfer beyond Ubuntu desktop tasks. On macOSWorld, MMSkills improve the completed large-model runs, including Gemini 3 Flash and GLM-5V, while VAB-Minecraft shows consistent gains in both success rate and average score across all evaluated models. Super Mario Bros follows the same pattern in the completed runs, with higher total performance and reward under MMSkills. These results indicate that MMSkills are not specialized to a single GUI benchmark; the same state-conditioned skill format helps in visually grounded game settings where recurring states and action strategies can be reused.

Table 1 OSWorld application-level success rates. All entries are percentages. "Calc", "Impress", and "Writer" denote LibreOffice applications.

Base model	Skill condition	Chrome	GIMP	Calc	Impress	Writer	Multi-app	OS	Mail	VLC	VS Code	Overall
Gemini 3.1 Pro	No skill	53.47	34.62	**57.45**	40.43	47.82	**31.97**	54.17	40.00	35.29	56.52	44.08
	Text-only	44.35	34.62	38.30	40.34	56.52	22.38	**70.83**	**66.67**	41.18	56.52	40.76
	MMSkills	**59.91**	**50.00**	53.19	**53.19**	**60.86**	24.11	**70.83**	**66.67**	**70.59**	**65.22**	**50.11**
Gemini 3 Flash	No skill	37.78	50.00	38.30	29.73	52.17	21.51	54.17	66.67	52.39	47.83	36.65
	Text-only	51.02	23.08	38.30	34.00	56.52	19.16	54.17	60.00	58.82	52.17	40.27
	MMSkills	**55.37**	42.31	**53.19**	**40.34**	**56.52**	**30.98**	**75.00**	**66.67**	52.94	**60.87**	**47.97**
Qwen3-VL-235B	No skill	15.56	38.46	17.02	25.53	43.48	9.48	25.00	26.67	17.65	34.78	21.34
	Text-only	42.22	50.00	10.64	21.31	34.78	14.86	33.33	60.00	35.29	47.83	28.57
	MMSkills	**59.91**	**69.23**	**23.40**	**32.01**	**47.82**	**19.35**	**41.67**	**73.33**	**41.18**	**56.52**	**39.17**
GLM-5V	No skill	37.78	19.23	21.28	29.70	26.08	18.70	54.17	53.33	11.76	47.83	28.71
	Text-only	**53.24**	**53.85**	**31.91**	**31.98**	**52.17**	20.24	20.83	**46.67**	**35.29**	**65.22**	36.61
	MMSkills	51.02	**53.85**	**31.91**	31.83	43.47	**22.26**	**66.67**	40.00	23.53	**65.22**	**38.51**
Kimi-K2.6	No skill	51.02	34.62	34.04	35.32	30.43	14.86	54.17	66.67	32.60	52.17	34.98
	Text-only	**57.69**	40.00	**40.43**	36.14	17.38	22.38	62.50	53.33	**58.82**	43.48	39.66
	MMSkills	**57.69**	**42.31**	**40.43**	**48.92**	**60.86**	**23.40**	**79.17**	**73.33**	41.18	**69.57**	**46.59**
Qwen3-VL-8B-Instruct	No skill	15.47	7.69	2.13	8.59	4.34	7.33	25.00	13.33	29.41	17.39	10.78
	Text-only	19.91	11.54	6.38	16.99	**17.39**	7.33	16.67	33.33	17.65	34.78	14.93
	MMSkills	**39.91**	**42.31**	**8.51**	**23.37**	**17.39**	**13.43**	**25.00**	**60.00**	**29.41**	**47.83**	**25.40**

Note: Due to the substantially higher inference cost and wall-clock time of Gemini 3.1 Pro and Kimi-K2.6, we report their full three-condition results only on OSWorld.

3.3 RQ2: Ablations of Skill Content and Branch Loading

Figure 3 combines the skill-content and branch-loading ablations. Unless otherwise stated, skill variants use the branch-loaded agent; the main exception is *Direct load*, which inserts skill content into the main context. For skill content, we compare text-only skills, MMSkills without state cards, MMSkills without images, and the complete MMSkills package. **State cards and multi-view visual evidence both improve skill utility.** Text-only branch loading already improves over the no-skill baseline, but the complete MMSkills package is consistently stronger. Removing state cards weakens the agent's ability to distinguish relevant runtime states, while removing images preserves decision rules but removes visual grounding evidence. Both removals reduce performance on OSWorld and VAB-Minecraft, confirming that state cards and keyframes play complementary roles: one supports state discrimination, and the other helps the agent recognize the corresponding visual evidence. **Branch loading helps even for text-only skills.** The branch-loaded text-only variant is stronger than direct text loading in most model–benchmark pairs, indicating that the temporary branch improves skill interpretation even before multimodal evidence is introduced.

For branch loading, we ablate whether skill evidence is inspected in a temporary branch and whether Stage-1 view selection filters state cards and keyframes. **Branch loading and view selection address different failure modes.** Direct-full loading hurts performance because unfiltered images and state descriptions pollute the main context; view selection alone reduces this damage but stays near baseline. Branch loading already gives clear gains, and the full two-stage design performs best, indicating that separated evidence inspection and filtered visual evidence are both necessary.

3.4 RQ3: Skill Usage and Interaction Dynamics

Table 3 analyzes when and how agents call skills. **MMSkills are invoked more often than text-only skills.** Invocation coverage increases on both OSWorld and VAB-Minecraft for Gemini 3 Flash and Qwen3-VL-235B, with the largest OSWorld change rising from 37.50% to 65.28% for Qwen3-VL-235B. This suggests that multimodal skills make external knowledge easier to recognize as relevant: state cards expose when-to-use and when-not-to-use conditions, and visual cues help the agent detect when its current observation matches a reusable procedural state.

MMSkills shorten trajectories rather than merely adding extra consultation. Text-only skills can add overhead when they provide procedural hints without visual grounding, but MMSkills reduce average steps in every setting, with the

Table 2 Auxiliary GUI and game-based visual-agent results. macOSWorld reports domain-level and overall success rates; VAB-Minecraft reports success rate and average score; Super Mario Bros reports total performance and total reward.

		macOSWorld						VAB-Minecraft		Super Mario Bros	
Base model	Skill condition	File	Media	Prod.	Sys/IF	Apps	Overall	Success	Avg. score	Total perf.	Total reward
Gemini 3 Flash	No skill	41.38	33.33	60.00	62.07	55.79	55.94	67.24	0.7462	411.00	766.67
	Text-only	31.03	25.00	62.86	**75.86**	55.26	53.85	68.96	0.7541	548.00	912.00
	MMSkills	**58.62**	**50.00**	**77.14**	65.52	**65.73**	**65.73**	**73.28**	**0.7884**	**624.00**	**1081.33**
Qwen3-VL-235B	No skill	31.03	**58.33**	51.43	58.62	44.74	47.55	52.59	0.6308	454.50	955.50
	Text-only	34.48	33.33	37.14	51.72	52.63	43.36	55.17	0.6634	610.50	1138.25
	MMSkills	**37.93**	33.33	**54.29**	**62.07**	**57.89**	**51.75**	**62.07**	**0.7114**	**788.00**	**1514.25**
GLM-5V	No skill	24.14	16.67	40.00	41.38	39.47	34.97	56.03	0.6701	612.75	1191.50
	Text-only	31.03	**66.67**	**62.86**	**58.62**	47.37	**51.75**	61.20	0.6938	794.50	1218.00
	MMSkills	**44.83**	**66.67**	48.57	**58.62**	**50.00**	**51.75**	**68.10**	**0.7495**	**950.50**	**1384.50**
Qwen3-VL-8B-Instruct	No skill	**10.34**	0.00	**14.29**	**3.45**	0.00	**6.29**	23.28	0.3017	415.25	928.75
	Text-only	0.00	**8.33**	2.86	**3.45**	**10.53**	4.90	29.31	0.3754	596.50	997.25
	MMSkills	6.90	**8.33**	8.57	**3.45**	5.26	**6.29**	**38.79**	**0.4668**	**764.00**	**1128.75**

Table 3 Skill invocation, interaction length, and selected views. "Invoked" is the percentage of cases with at least one skill call, and "Step Δ" is relative to the no-skill baseline.

Benchmark	Model	Skill condition	Invoked (%)	Calls/case	Steps	Step Δ	Views (Full/Focus/Before/After)
OSWorld	Gemini 3 Flash	No skill	–	–	13.11	0.00	–
		Text-only	41.11	0.7139	15.64	+2.53	–
		MMSkills	**62.50**	**0.9556**	**11.86**	**-1.25**	79/241/8/24
	Qwen3-VL-235B	No skill	–	–	15.22	0.00	–
		Text-only	37.50	0.4917	13.34	-1.88	–
		MMSkills	**65.28**	**0.9222**	**9.87**	**-5.35**	40/27/17/13
VAB-Minecraft	Gemini 3 Flash	No skill	–	–	16.92	0.00	–
		Text-only	68.97	1.8706	17.30	+0.38	–
		MMSkills	**81.90**	**2.4310**	**13.75**	**-3.17**	105/205/15/12
	Qwen3-VL-235B	No skill	–	–	34.74	0.00	–
		Text-only	54.31	1.5776	31.36	-3.38	–
		MMSkills	**64.66**	**2.3534**	**27.07**	**-7.67**	98/196/13/10

largest reductions appearing for Qwen3-VL-235B. These reductions indicate that multimodal skills help agents find shorter task-solving paths and avoid unnecessary exploration or repeated low-value actions. **Focus crops dominate selected visual evidence.** The branch does not load all views uniformly: focus crops are selected most frequently in three of four settings, while full-frame, before, and after views provide global context, transition evidence, and completion references when local crops alone are insufficient.

3.5 RQ4: Behavioral Shift Analysis

Figure 4 shows that the effect of MMSkills is not merely a success-rate gain. **MMSkills reduce low-level action load.** Gemini 3 Flash uses substantially fewer primitives per task, and Qwen3-VL-235B shows a similar reduction, especially in click actions. This supports the view that multimodal state cards and visual evidence constrain the agent's search space: the agent performs fewer exploratory GUI operations before reaching a useful state. **The behavioral shift is strongest for Qwen3-VL-235B.** Its click share drops from 75.8% to 63.7%, while keyboard and DONE actions increase, suggesting that MMSkills help click-heavy agents move toward more structured input and stronger completion judgments.

MMSkills suppress repetitive trajectories and improve completion awareness. The effect is clearest for Qwen3-VL-

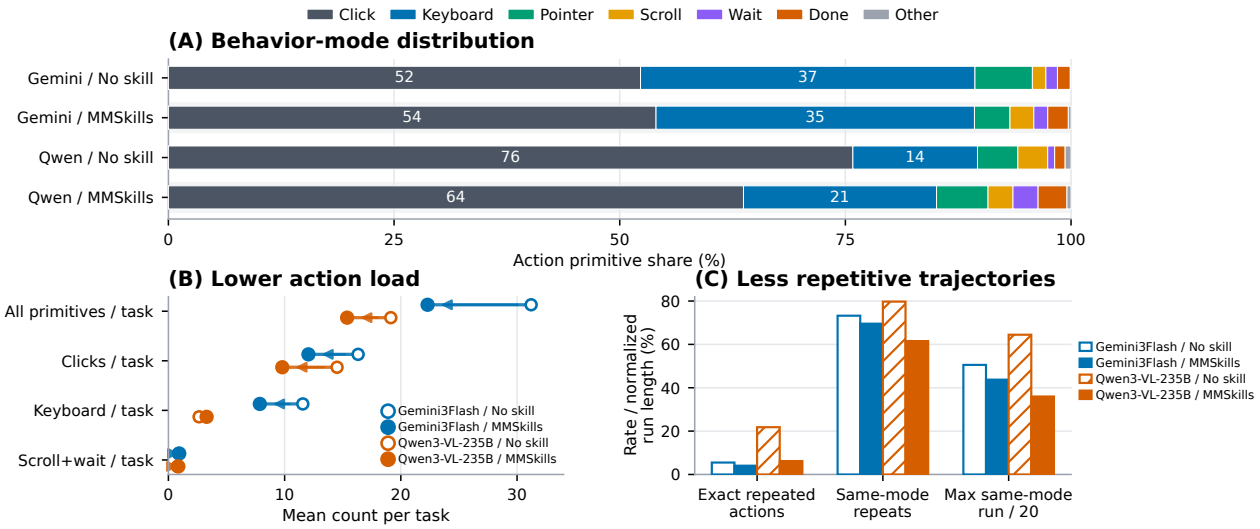

Figure 4 Behavioral shifts induced by MMSkills on OSWorld. Panel (A) reports the distribution of executed action primitives. Panel (B) compares the average number of low-level primitives per task. Panel (C) measures repetitive behavior through exact repeated actions, repeated action modes, and the longest same-mode run normalized by the 20-step budget.

235B: exact repeated actions fall from 21.8% to 6.2%, and the longest same-mode run decreases substantially. Gemini 3 Flash shows the same direction of change, though from a stronger baseline. MMSkills also increase DONE behavior for both models, indicating that state cards and verification cues help agents decide not only what to do next, but also when the task is complete. Overall, MMSkills reshape agent behavior from exploratory trial-and-error toward grounded, state-aware execution; Appendix F provides the GLM-5V and Kimi-K2.6 analysis.

4 Related Work

Skills for agents. Skill reuse has roots in temporal abstraction and motor primitives (Sutton et al., 1999; Ijspeert et al., 2013), and recent LLM agents store reusable behavior as language, code, APIs, or learned libraries (Ahn et al., 2022; Liang et al., 2023; Yao et al., 2023; Shinn et al., 2023; Wang et al., 2023a; Zheng et al., 2025; Chen et al., 2026; Wang et al., 2026a; Alzubi et al., 2026; Ma et al., 2026; Xia et al., 2026). A complementary line treats accumulated experience as long-term agent memory (Park et al., 2023; Packer et al., 2024), while surveys and benchmarks evaluate skill relevance, selection, and safety (Xu and Yan, 2026; Li et al., 2026b; Wang et al., 2026b; Liu et al., 2026). MMSkills follows this modular view but stores state-conditioned multimodal packages and uses branch loading instead of inserting full skill memory; Appendix J expands the discussion.

Visual agents. Visual-agent benchmarks span web, mobile, desktop, and embodied environments (Deng et al., 2023; Zhou et al., 2024; Koh et al., 2024; He et al., 2024; Rawles et al., 2025; Xie et al., 2024; Yang et al., 2025b; Liu et al., 2024a), and model and framework work improves screenshot grounding and GUI control (Cheng et al., 2024; Wu et al., 2024; Qin et al., 2025; Agashe et al., 2024; Hong et al., 2024; Zheng et al., 2024; Zhang et al., 2023; Lu et al., 2024). Dedicated grounding benchmarks measure how reliably models localize UI elements from instructions (Li et al., 2025a; Gou et al., 2025; Wang et al., 2025b; Xu et al., 2025). MMSkills builds on these capabilities but operates higher: it tells the agent which procedural state matters and what visual evidence confirms it.

Closest to our work, Mirage-1 introduces hierarchical multimodal skills, XSkill extracts skills from visually grounded experience, and CUA-Skill represents computer-use skills as parameterized procedures and execution graphs (Xie et al., 2025; Jiang et al., 2026; Chen et al., 2026). MMSkills differs by organizing skills around runtime state cards and multi-view evidence, and by using branch loading to align selected evidence with the live observation before the main agent acts.

5 Conclusion and Limitations

We introduced **MMSkills**, a framework that represents reusable skills for visual agents as multimodal procedural knowledge. By combining textual procedures, runtime state cards, multi-view keyframes, and branch-loaded use, MMSkills improve GUI and game-based visual agents across model families. The main limitations are dependence on source-trajectory coverage, possible errors from skill generation or visual grounding, and extra inference cost from branch loading. Extending MMSkills to broader embodied or safety-critical settings will require stronger verification and online skill repair.

References

Saaket Agashe, Jiuzhou Han, Shuyu Gan, Jiachen Yang, Ang Li, and Xin Eric Wang. Agent S: An open agentic framework that uses computers like a human, 2024. URL https://arxiv.org/abs/2410.08164.

Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Chuyuan Fu, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Daniel Ho, Jasmine Hsu, Julian Ibarz, Brian Ichter, Alex Irpan, Eric Jang, Rosario Jauregui Ruano, Kyle Jeffrey, Sally Jesmonth, Nikhil J Joshi, Ryan Julian, Dmitry Kalashnikov, Yuheng Kuang, Kuang-Huei Lee, Sergey Levine, Yao Lu, Linda Luu, Carolina Parada, Peter Pastor, Jornell Quiambao, Kanishka Rao, Jarek Rettinghouse, Diego Reyes, Pierre Sermanet, Nicolas Sievers, Clayton Tan, Alexander Toshev, Vincent Vanhoucke, Fei Xia, Ted Xiao, Peng Xu, Sichun Xu, Mengyuan Yan, and Andy Zeng. Do as i can, not as i say: Grounding language in robotic affordances, 2022. URL https://arxiv.org/abs/2204.01691.

Salaheddin Alzubi, Noah Provenzano, Jaydon Bingham, Weiyuan Chen, and Tu Vu. Evoskill: Automated skill discovery for multi-agent systems, 2026. URL https://arxiv.org/abs/2603.02766.

Shuai Bai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, Xionghui Chen, Zesen Cheng, Lianghao Deng, Wei Ding, Chang Gao, Chunjiang Ge, Wenbin Ge, Zhifang Guo, Qidong Huang, Jie Huang, Fei Huang, Binyuan Hui, Shutong Jiang, Zhaohai Li, Mingsheng Li, Mei Li, Kaixin Li, Zicheng Lin, Junyang Lin, Xuejing Liu, Jiawei Liu, Chenglong Liu, Yang Liu, Dayiheng Liu, Shixuan Liu, Dunjie Lu, Ruilin Luo, Chenxu Lv, Rui Men, Lingchen Meng, Xuancheng Ren, Xingzhang Ren, Sibo Song, Yuchong Sun, Jun Tang, Jianhong Tu, Jianqiang Wan, Peng Wang, Pengfei Wang, Qiuyue Wang, Yuxuan Wang, Tianbao Xie, Yiheng Xu, Haiyang Xu, Jin Xu, Zhibo Yang, Mingkun Yang, Jianxin Yang, An Yang, Bowen Yu, Fei Zhang, Hang Zhang, Xi Zhang, Bo Zheng, Humen Zhong, Jingren Zhou, Fan Zhou, Jing Zhou, Yuanzhi Zhu, and Ke Zhu. Qwen3-VL technical report, 2025. URL https://arxiv.org/abs/2511.21631.

Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. Longbench: A bilingual, multitask benchmark for long context understanding, 2024. URL https://arxiv.org/abs/2308.14508.

Tianyi Chen, Yinheng Li, Michael Solodko, Sen Wang, Nan Jiang, Tingyuan Cui, Junheng Hao, Jongwoo Ko, Sara Abdali, Leon Xu, Suzhen Zheng, Hao Fan, Pashmina Cameron, Justin Wagle, and Kazuhito Koishida. CUA-skill: Develop skills for computer using agent, 2026. URL https://arxiv.org/abs/2601.21123.

Kanzhi Cheng, Qiushi Sun, Yougang Chu, Fangzhi Xu, Yantao Li, Jianbing Zhang, and Zhiyong Wu. Seeclick: Harnessing GUI grounding for advanced visual GUI agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, pages 9313–9332. Association for Computational Linguistics, 2024. doi: 10.18653/V1/2024.ACL-LONG.505. URL https://doi.org/10.18653/v1/2024.acl-long.505.

Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Samuel Stevens, Boshi Wang, Huan Sun, and Yu Su. Mind2web: Towards a generalist agent for the web, 2023. URL https://arxiv.org/abs/2306.06070.

Boyu Gou, Ruohan Wang, Boyuan Zheng, Yanan Xie, Cheng Chang, Yiheng Shu, Huan Sun, and Yu Su. Navigating the digital world as humans do: Universal visual grounding for gui agents, 2025. URL https://arxiv.org/abs/2410.05243.

Hongliang He, Wenlin Yao, Kaixin Ma, Wenhao Yu, Yong Dai, Hongming Zhang, Zhenzhong Lan, and Dong Yu. Webvoyager: Building an end-to-end web agent with large multimodal models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, pages 6864–6890. Association for Computational Linguistics, 2024. doi: 10.18653/V1/2024.ACL-LONG.371. URL https://doi.org/10.18653/v1/2024.acl-long.371.

Wenyi Hong, Weihan Wang, Qingsong Lv, Jiazheng Xu, Wenmeng Yu, Junhui Ji, Yan Wang, Zihan Wang, Yuxuan Zhang, Juanzi Li, Bin Xu, Yuxiao Dong, Ming Ding, and Jie Tang. Cogagent: A visual language model for gui agents, 2024. URL https://arxiv.org/abs/2312.08914.

Lanxiang Hu, Mingjia Huo, Yuxuan Zhang, Haoyang Yu, Eric P. Xing, Ion Stoica, Tajana Rosing, Haojian Jin, and Hao Zhang. lmgame-bench: How good are llms at playing games?, 2025. URL https://arxiv.org/abs/2505.15146.

Auke Jan Ijspeert, Jun Nakanishi, Heiko Hoffmann, Peter Pastor, and Stefan Schaal. Dynamical movement primitives: Learning attractor models for motor behaviors. *Neural Computation*, 25(2):328–373, 2013. doi: 10.1162/NECO_a_00393. URL https://doi.org/10.1162/NECO_a_00393.

Guanyu Jiang, Zhaochen Su, Xiaoye Qu, and Yi R. Fung. Xskill: Continual learning from experience and skills in multimodal agents, 2026. URL https://arxiv.org/abs/2603.12056.

Jing Yu Koh, Robert Lo, Lawrence Jang, Vikram Duvvur, Ming Chong Lim, Po-Yu Huang, Graham Neubig, Shuyan Zhou, Russ Salakhutdinov, and Daniel Fried. Visualwebarena: Evaluating multimodal agents on realistic visual web tasks. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, pages 881–905. Association for Computational Linguistics, 2024. doi: 10.18653/V1/2024.ACL-LONG.50. URL https://doi.org/10.18653/v1/2024.acl-long.50.

Kaixin Li, Ziyang Meng, Hongzhan Lin, Ziyang Luo, Yuchen Tian, Jing Ma, Zhiyong Huang, and Tat-Seng Chua. Screenspot-pro: Gui grounding for professional high-resolution computer use, 2025a. URL https://arxiv.org/abs/2504.07981.

Qingyao Li, Wei Xia, Kounianhua Du, Xinyi Dai, Ruiming Tang, Yasheng Wang, Yong Yu, and Weinan Zhang. Rethinkmcts: Refining erroneous thoughts in monte carlo tree search for code generation, 2025b. URL https://arxiv.org/abs/2409.09584.

Qingyao Li, Xinyi Dai, Weiwen Liu, Xiangyang Li, Yasheng Wang, Ruiming Tang, Yong Yu, and Weinan Zhang. ATGen: Adversarial reinforcement learning for test case generation. In *The Fourteenth International Conference on Learning Representations*, 2026a. URL https://openreview.net/forum?id=Sxj4o3qXtl.

Xiangyi Li, Wenbo Chen, Yimin Liu, Shenghan Zheng, Xiaokun Chen, Yifeng He, Yubo Li, Bingran You, Haotian Shen, Jiankai Sun, Shuyi Wang, Binxu Li, Qunhong Zeng, Di Wang, Xuandong Zhao, Yuanli Wang, Roey Ben Chaim, Zonglin Di, Yipeng Gao, Junwei He, Yizhuo He, Liqiang Jing, Luyang Kong, Xin Lan, Jiachen Li, Songlin Li, Yijiang Li, Yueqian Lin, Xinyi Liu, Xuanqing Liu, Haoran Lyu, Ze Ma, Bowei Wang, Runhui Wang, Tianyu Wang, Wengao Ye, Yue Zhang, Hanwen Xing, Yiqi Xue, Steven Dillmann, and Han chung Lee. Skillsbench: Benchmarking how well agent skills work across diverse tasks, 2026b. URL https://arxiv.org/abs/2602.12670.

Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. Code as policies: Language model programs for embodied control. In *IEEE International Conference on Robotics and Automation, ICRA 2023*, pages 9493–9500. IEEE, 2023. doi: 10.1109/ICRA48891.2023.10160591. URL https://doi.org/10.1109/ICRA48891.2023.10160591.

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts, 2023. URL https://arxiv.org/abs/2307.03172.

Xiao Liu, Tianjie Zhang, Yu Gu, Iat Long Iong, Yifan Xu, Xixuan Song, Shudan Zhang, Hanyu Lai, Xinyi Liu, Hanlin Zhao, Jiadai Sun, Xinyue Yang, Yu Yang, Zehan Qi, Shuntian Yao, Xueqiao Sun, Siyi Cheng, Qinkai Zheng, Hao Yu, Hanchen Zhang, Wenyi Hong, Ming Ding, Lihang Pan, Xiaotao Gu, Aohan Zeng, Zhengxiao Du, Chan Hee Song, Yu Su, Yuxiao Dong, and Jie Tang. Visualagentbench: Towards large multimodal models as visual foundation agents, 2024a. URL https://arxiv.org/abs/2408.06327.

Yifan Liu, Kangning Zhang, Xiangyuan Ren, Yanhua Huang, Jiarui Jin, Yingjie Qin, Ruilong Su, Ruiwen Xu, Yong Yu, and Weinan Zhang. Alignrec: Aligning and training in multimodal recommendations. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*, CIKM '24, pages 1503–1512, New York, NY, USA, 2024b. Association for Computing Machinery. ISBN 9798400704369. doi: 10.1145/3627673.3679626. URL https://doi.org/10.1145/3627673.3679626.

Yujian Liu, Jiabao Ji, Li An, Tommi Jaakkola, Yang Zhang, and Shiyu Chang. How well do agentic skills work in the wild: Benchmarking LLM skill usage in realistic settings, 2026. URL https://arxiv.org/abs/2604.04323.

Yadong Lu, Jianwei Yang, Yelong Shen, and Ahmed Awadallah. Omniparser for pure vision based gui agent, 2024. URL https://arxiv.org/abs/2408.00203.

Ziyu Ma, Shidong Yang, Yuxiang Ji, Xucong Wang, Yong Wang, Yiming Hu, Tongwen Huang, and Xiangxiang Chu. Skillclaw: Let skills evolve collectively with agentic evolver, 2026. URL https://arxiv.org/abs/2604.08377.

Richard E. Mayer. *Multimedia Learning*. Cambridge University Press, 2009. doi: 10.1017/CBO9780511811678. URL https://doi.org/10.1017/CBO9780511811678.

Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. Memgpt: Towards llms as operating systems, 2024. URL https://arxiv.org/abs/2310.08560.

Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior, 2023. URL https://arxiv.org/abs/2304.03442.

Yujia Qin, Yining Ye, Junjie Fang, Haoming Wang, Shihao Liang, Shizuo Tian, Junda Zhang, Jiahao Li, Yunxin Li, Shijue Huang, Wanjun Zhong, Kuanye Li, Jiale Yang, Yu Miao, Woyu Lin, Longxiang Liu, Xu Jiang, Qianli Ma, Jingyu Li, Xiaojun Xiao, Kai Cai, Chuang Li, Yaowei Zheng, Chaolin Jin, Chen Li, Xiao Zhou, Minchao Wang, Haoli Chen, Zhaojian Li, Haihua Yang, Haifeng

Liu, Feng Lin, Tao Peng, Xin Liu, and Guang Shi. UI-TARS: Pioneering automated GUI interaction with native agents, 2025. URL https://arxiv.org/abs/2501.12326.

Christopher Rawles, Alice Li, Daniel Rodriguez, Oriana Riva, and Timothy Lillicrap. Android in the wild: A large-scale dataset for android device control, 2023. URL https://arxiv.org/abs/2307.10088.

Christopher Rawles, Sarah Clinckemaillie, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William Bishop, Wei Li, Folawiyo Campbell-Ajala, Daniel Toyama, Robert Berry, Divya Tyamagundlu, Timothy Lillicrap, and Oriana Riva. Androidworld: A dynamic benchmarking environment for autonomous agents, 2025. URL https://arxiv.org/abs/2405.14573.

Shuai Shao, Yixiang Liu, Bingwei Lu, and Weinan Zhang. Monoscale: Scaling multi-agent system with monotonic improvement, 2026a. URL https://arxiv.org/abs/2601.23219.

Shuai Shao, Qihan Ren, Chen Qian, Boyi Wei, Dadi Guo, Jingyi Yang, Xinhao Song, Linfeng Zhang, Weinan Zhang, Dongrui Liu, and Jing Shao. Your agent may misevolve: Emergent risks in self-evolving llm agents, 2026b. URL https://arxiv.org/abs/2509.26354.

Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems 36*, 2023. URL http://papers.nips.cc/paper_files/paper/2023/hash/1b44b878bb782e6954cd888628510e90-Abstract-Conference.html.

Richard S. Sutton, Doina Precup, and Satinder Singh. Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, 112(1–2):181–211, 1999. doi: 10.1016/S0004-3702(99)00052-1. URL https://doi.org/10.1016/S0004-3702(99)00052-1.

Kimi Team, Tongtong Bai, Yifan Bai, Yiping Bao, S. H. Cai, Yuan Cao, Y. Charles, H. S. Che, Cheng Chen, Guanduo Chen, Huarong Chen, Jia Chen, Jiahao Chen, Jianlong Chen, Jun Chen, Kefan Chen, Liang Chen, Ruijue Chen, Xinhao Chen, Yanru Chen, Yanxu Chen, Yicun Chen, Yimin Chen, Yingjiang Chen, Yuankun Chen, Yujie Chen, Yutian Chen, Zhirong Chen, Ziwei Chen, Dazhi Cheng, Minghan Chu, Jialei Cui, Jiaqi Deng, Muxi Diao, Hao Ding, Mengfan Dong, Mengnan Dong, Yuxin Dong, Yuhao Dong, Angang Du, Chenzhuang Du, Dikang Du, Lingxiao Du, Yulun Du, Yu Fan, Shengjun Fang, Qiulin Feng, Yichen Feng, Garimugai Fu, Kelin Fu, Hongcheng Gao, Tong Gao, Yuyao Ge, Shangyi Geng, Chengyang Gong, Xiaochen Gong, Zhuoma Gongque, Qizheng Gu, Xinran Gu, Yicheng Gu, Longyu Guan, Yuanying Guo, Xiaoru Hao, Weiran He, Wenyang He, Yunjia He, Chao Hong, Hao Hu, Jiaxi Hu, Yangyang Hu, Zhenxing Hu, Ke Huang, Ruiyuan Huang, Weixiao Huang, Zhiqi Huang, Tao Jiang, Zhejun Jiang, Xinyi Jin, Yu Jing, Guokun Lai, Aidi Li, C. Li, Cheng Li, Fang Li, Guanghe Li, Guanyu Li, Haitao Li, Haoyang Li, Jia Li, Jingwei Li, Junxiong Li, Lincan Li, Mo Li, Weihong Li, Wentao Li, Xinhang Li, Xinhao Li, Yang Li, Yanhao Li, Yiwei Li, Yuxiao Li, Zhaowei Li, Zheming Li, Weilong Liao, Jiawei Lin, Xiaohan Lin, Zhishan Lin, Zichao Lin, Cheng Liu, Chenyu Liu, Hongzhang Liu, Liang Liu, Shaowei Liu, Shudong Liu, Shuran Liu, Tianwei Liu, Tianyu Liu, Weizhou Liu, Xiangyan Liu, Yangyang Liu, Yanming Liu, Yibo Liu, Yuanxin Liu, Yue Liu, Zhengying Liu, Zhongnuo Liu, Enzhe Lu, Haoyu Lu, Zhiyuan Lu, Junyu Luo, Tongxu Luo, Yashuo Luo, Long Ma, Yingwei Ma, Shaoguang Mao, Yuan Mei, Xin Men, Fanqing Meng, Zhiyong Meng, Yibo Miao, Minqing Ni, Kun Ouyang, Siyuan Pan, Bo Pang, Yuchao Qian, Ruoyu Qin, Zeyu Qin, Jiezhong Qiu, Bowen Qu, Zeyu Shang, Youbo Shao, Tianxiao Shen, Zhennan Shen, Juanfeng Shi, Lidong Shi, Shengyuan Shi, Feifan Song, Pengwei Song, Tianhui Song, Xiaoxi Song, Hongjin Su, Jianlin Su, Zhaochen Su, Lin Sui, Jinsong Sun, Junyao Sun, Tongyu Sun, Flood Sung, Yunpeng Tai, Chuning Tang, Heyi Tang, Xiaojuan Tang, Zhengyang Tang, Jiawen Tao, Shiyuan Teng, Chaoran Tian, Pengfei Tian, Ao Wang, Bowen Wang, Chensi Wang, Chuang Wang, Congcong Wang, Dingkun Wang, Dinglu Wang, Dongliang Wang, Feng Wang, Hailong Wang, Haiming Wang, Hengzhi Wang, Huaqing Wang, Hui Wang, Jiahao Wang, Jinhong Wang, Jiuzheng Wang, Kaixin Wang, Linian Wang, Qibin Wang, Shengjie Wang, Shuyi Wang, Si Wang, Wei Wang, Xiaochen Wang, Xinyuan Wang, Yao Wang, Yejie Wang, Yipu Wang, Yiqin Wang, Yucheng Wang, Yuzhi Wang, Zhaoji Wang, Zhaowei Wang, Zhengtao Wang, Zhexu Wang, Zihan Wang, Zizhe Wang, Chu Wei, Ming Wei, Chuan Wen, Zichen Wen, Chengjie Wu, Haoning Wu, Junyan Wu, Rucong Wu, Wenhao Wu, Yuefeng Wu, Yuhao Wu, Yuxin Wu, Zijian Wu, Chenjun Xiao, Jin Xie, Xiaotong Xie, Yuchong Xie, Yifei Xin, Bowei Xing, Boyu Xu, Jianfan Xu, Jing Xu, Jinjing Xu, L. H. Xu, Lin Xu, Suting Xu, Weixin Xu, Xinbo Xu, Xinran Xu, Yangchuan Xu, Yichang Xu, Yuemeng Xu, Zelai Xu, Ziyao Xu, Junjie Yan, Yuzi Yan, Guangyao Yang, Hao Yang, Junwei Yang, Kai Yang, Ningyuan Yang, Ruihan Yang, Xiaofei Yang, Xinlong Yang, Ying Yang, Yi Yang, Yi Yang, Zhen Yang, Zhilin Yang, Zonghan Yang, Haotian Yao, Dan Ye, Wenjie Ye, Zhuorui Ye, Bohong Yin, Chengzhen Yu, Longhui Yu, Tao Yu, Tianxiang Yu, Enming Yuan, Mengjie Yuan, Xiaokun Yuan, Yang Yue, Weihao Zeng, Dunyuan Zha, Haobing Zhan, Dehao Zhang, Hao Zhang, Jin Zhang, Puqi Zhang, Qiao Zhang, Rui Zhang, Xiaobin Zhang, Y. Zhang, Yadong Zhang, Yangkun Zhang, Yichi Zhang, Yizhi Zhang, Yongting Zhang, Yu Zhang, Yushun Zhang, Yutao Zhang, Yutong Zhang, Zheng Zhang, Chenguang Zhao, Feifan Zhao, Jinxiang Zhao, Shuai Zhao, Xiangyu Zhao, Yikai Zhao, Zijia Zhao, Huabin Zheng, Ruihan Zheng, Shaojie Zheng, Tengyang Zheng, Junfeng Zhong, Longguang Zhong, Weiming Zhong, M. Zhou, Runjie Zhou, Xinyu Zhou, Zaida Zhou, Jinguo Zhu, Liya Zhu, Xinhao Zhu, Yuxuan Zhu, Zhen Zhu, Jingze Zhuang, Weiyu Zhuang, Ying Zou, and Xinxing Zu. Kimi k2.5: Visual agentic intelligence, 2026a. URL https://arxiv.org/abs/2602.02276.

V Team, Wenyi Hong, Xiaotao Gu, Ziyang Pan, Zhen Yang, Yuting Wang, Yue Wang, Yuanchang Yue, Yu Wang, Yanling Wang, Yan Wang, Xijun Liu, Wenmeng Yu, Weihan Wang, Wei Li, Shuaiqi Duan, Sheng Yang, Ruiliang Lv, Mingdao Liu, Lihang Pan,

Ke Ning, Junhui Ji, Jinjiang Wang, Jing Chen, Jiazheng Xu, Jiale Zhu, Jiale Cheng, Ji Qi, Guobing Gan, Guo Wang, Cong Yao, Zijun Dou, Zihao Zhou, Zihan Wang, Zhiqi Ge, Zhijie Li, Zhenyu Hou, Zhao Xue, Zehui Wang, Zehan Qi, Zehai He, Yutao Zhang, Yusen Liu, Yukuo Cen, Yuchen Li, Yuan Wang, Yu Yang, Yongbin Liu, Yijian Lu, Yifan Xu, Yanzi Wang, Yanxiao Zhao, Yanfeng Wang, Yadong Xue, Yabo Xu, Xinyu Zhang, Xinyu Liu, Xiao Liu, Wenyi Zhao, Wenkai Li, Tianyu Tong, Tianshu Zhang, Shudan Zhang, Shengdong Yan, Qinkai Zheng, Mingde Xu, Licheng Bao, lat Long long, Jiaxing Xu, Jiaxin Fan, Jiawen Qian, Jiali Chen, Jiahui Lin, Jiadai Sun, Haozhi Zheng, Haoran Wang, Haochen Li, Hanyu Liu, Han Xu, Fan Yang, Dan Zhang, Da Yin, Chuangxin Zhao, Chengcheng Wu, Boyan Shi, Bowen Lv, Bowei Jia, Bo Li, Bin Chen, Baoxu Wang, Peng Zhang, Debing Liu, Bin Xu, Juanzi Li, Minlie Huang, Yuxiao Dong, and Jie Tang. Glm-5v-turbo: Toward a native foundation model for multimodal agents, 2026b. URL https://arxiv.org/abs/2604.26752.

Chenxi Wang, Zhuoyun Yu, Xin Xie, Wuguannan Yao, Runnan Fang, Shuofei Qiao, Kexin Cao, Guozhou Zheng, Xiang Qi, Peng Zhang, and Shumin Deng. Skillx: Automatically constructing skill knowledge bases for agents, 2026a. URL https://arxiv.org/abs/2604.04804.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models, 2023a. URL https://arxiv.org/abs/2305.16291.

Leye Wang, Zixing Wang, and Anjie Xu. Skilltester: Benchmarking utility and security of agent skills, 2026b. URL https://arxiv.org/abs/2603.28815.

Shijian Wang, Jiarui Jin, Runhao Fu, Zexuan Yan, Xingjian Wang, Mengkang Hu, Eric Wang, Xiaoxi Li, Kangning Zhang, Li Yao, Wenxiang Jiao, Xuelian Cheng, Yuan Lu, and Zongyuan Ge. Museagent: A multimodal reasoning agent with stateful experiences, 2026c. URL https://arxiv.org/abs/2603.27813.

Xinyuan Wang, Bowen Wang, Dunjie Lu, Junlin Yang, Tianbao Xie, Junli Wang, Jiaqi Deng, Xiaole Guo, Yiheng Xu, Chen Henry Wu, Zhennan Shen, Zhuokai Li, Ryan Li, Xiaochuan Li, Junda Chen, Boyuan Zheng, Peihang Li, Fangyu Lei, Ruisheng Cao, Yeqiao Fu, Dongchan Shin, Martin Shin, Jiarui Hu, Yuyan Wang, Jixuan Chen, Yuxiao Ye, Danyang Zhang, Dikang Du, Hao Hu, Huarong Chen, Zaida Zhou, Haotian Yao, Ziwei Chen, Qizheng Gu, Yipu Wang, Heng Wang, Diyi Yang, Victor Zhong, Flood Sung, Y. Charles, Zhilin Yang, and Tao Yu. Opencua: Open foundations for computer-use agents, 2025a. URL https://arxiv.org/abs/2508.09123.

Xuehui Wang, Zhenyu Wu, JingJing Xie, Zichen Ding, Bowen Yang, Zehao Li, Zhaoyang Liu, Qingyun Li, Xuan Dong, Zhe Chen, Weiyun Wang, Xiangyu Zhao, Jixuan Chen, Haodong Duan, Tianbao Xie, Chenyu Yang, Shiqian Su, Yue Yu, Yuan Huang, Yiqian Liu, Xiao Zhang, Yanting Zhang, Xiangyu Yue, Weijie Su, Xizhou Zhu, Wei Shen, Jifeng Dai, and Wenhai Wang. Mmbench-gui: Hierarchical multi-platform evaluation framework for gui agents, 2025b. URL https://arxiv.org/abs/2507.19478.

Zihao Wang, Shaofei Cai, Anji Liu, Yonggang Jin, Jinbing Hou, Bowei Zhang, Haowei Lin, Zhaofeng He, Zilong Zheng, Yaodong Yang, Xiaojian Ma, and Yitao Liang. Jarvis-1: Open-world multi-task agents with memory-augmented multimodal language models, 2023b. URL https://arxiv.org/abs/2311.05997.

Zihao Wang, Shaofei Cai, Guanzhou Chen, Anji Liu, Xiaojian Ma, and Yitao Liang. Describe, explain, plan and select: Interactive planning with large language models enables open-world multi-task agents, 2024. URL https://arxiv.org/abs/2302.01560.

Zhiyong Wu, Zhenyu Wu, Fangzhi Xu, Yian Wang, Qiushi Sun, Chengyou Jia, Kanzhi Cheng, Zichen Ding, Liheng Chen, Paul Pu Liang, and Yu Qiao. OS-ATLAS: A foundation action model for generalist GUI agents, 2024. URL https://arxiv.org/abs/2410.23218.

Peng Xia, Jianwen Chen, Hanyang Wang, Jiaqi Liu, Kaide Zeng, Yu Wang, Siwei Han, Yiyang Zhou, Xujiang Zhao, Haifeng Chen, Zeyu Zheng, Cihang Xie, and Huaxiu Yao. Skillrl: Evolving agents via recursive skill-augmented reinforcement learning, 2026. URL https://arxiv.org/abs/2602.08234.

Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments, 2024. URL https://arxiv.org/abs/2404.07972.

Yuquan Xie, Zaijing Li, Rui Shao, Gongwei Chen, Kaiwen Zhou, Yinchuan Li, Dongmei Jiang, and Liqiang Nie. Mirage-1: Augmenting and updating GUI agent with hierarchical multimodal skills, 2025. URL https://arxiv.org/abs/2506.10387.

Renjun Xu and Yang Yan. Agent skills for large language models: Architecture, acquisition, security, and the path forward, 2026. URL https://arxiv.org/abs/2602.12430.

Yibin Xu, Liang Yang, Hao Chen, Hua Wang, Zhi Chen, and Yaohua Tang. Deskvision: Large scale desktop region captioning for advanced gui agents, 2025. URL https://arxiv.org/abs/2503.11170.

Jingyi Yang, Shuai Shao, Dongrui Liu, and Jing Shao. Riosworld: Benchmarking the risk of multimodal computer-use agents, 2025a. URL https://arxiv.org/abs/2506.00618.

Pei Yang, Hai Ci, and Mike Zheng Shou. macosworld: A multilingual interactive benchmark for GUI agents, 2025b. URL https://arxiv.org/abs/2506.04135.

Yingxuan Yang, Huacan Chai, Shuai Shao, Yuanyi Song, Siyuan Qi, Renting Rui, and Weinan Zhang. Agentnet: Decentralized evolutionary coordination for llm-based multi-agent systems, 2025c. URL https://arxiv.org/abs/2504.00587.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations, ICLR 2023*. OpenReview.net, 2023. URL https://openreview.net/forum?id=WE_vluYUL-X.

Chi Zhang, Zhao Yang, Jiaxuan Liu, Yucheng Han, Xin Chen, Zebiao Huang, Bin Fu, and Gang Yu. Appagent: Multimodal agents as smartphone users, 2023. URL https://arxiv.org/abs/2312.13771.

Kangning Zhang, Yingjie Qin, Jiarui Jin, Yifan Liu, Ruilong Su, Weinan Zhang, and Yong Yu. Dream: A dual representation learning model for multimodal recommendation, 2024. URL https://arxiv.org/abs/2404.11119.

Kangning Zhang, Wenxiang Jiao, Kounianhua Du, Yuan Lu, Weiwen Liu, Weinan Zhang, and Yong Yu. Looptool: Closing the data-training loop for robust llm tool calls, 2025. URL https://arxiv.org/abs/2511.09148.

Boyuan Zheng, Boyu Gou, Jihyung Kil, Huan Sun, and Yu Su. Gpt-4v(ision) is a generalist web agent, if grounded, 2024. URL https://arxiv.org/abs/2401.01614.

Boyuan Zheng, Michael Y. Fatemi, Xiaolong Jin, Zora Zhiruo Wang, Apurva Gandhi, Yueqi Song, Yu Gu, Jayanth Srinivasa, Gaowen Liu, Graham Neubig, and Yu Su. Skillweaver: Web agents can self-improve by discovering and honing skills, 2025. URL https://arxiv.org/abs/2504.07079.

Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. Webarena: A realistic web environment for building autonomous agents, 2024. URL https://arxiv.org/abs/2307.13854.

Appendix

A Benchmark Statistics

We use four visual-agent benchmarks. **OSWorld** is the primary GUI benchmark and contains Ubuntu desktop tasks across browsers, office software, creative tools, media applications, system settings, code editors, email, and multi-application workflows (Xie et al., 2024). **macOSWorld** provides an auxiliary cross-operating-system GUI evaluation with file management, media, productivity, system/interface, and system-application tasks (Yang et al., 2025b). **VAB-Minecraft** is the Minecraft subset of VisualAgentBench and evaluates item-acquisition tasks that require visual grounding, inventory tracking, recipe reasoning, tool use, and handling failed actions (Liu et al., 2024a). **LMGame-Bench** evaluates game-playing agents through a unified interface (Hu et al., 2025); we use Super Mario Bros because its recurring visual situations naturally align with reusable multimodal skills.

Table 4 Test-case distributions for OSWorld and macOSWorld. OSWorld contains 360 test cases; macOSWorld contains 143 test cases. "Share" is the percentage of test cases in each domain within the corresponding benchmark.

Benchmark	Domain	Count	Share	Snapshot-en	Snapshot-apps
OSWorld	Multi-app	93	25.83	–	–
OSWorld	LibreOffice Calc	47	13.06	–	–
OSWorld	LibreOffice Impress	47	13.06	–	–
OSWorld	Chrome	45	12.50	–	–
OSWorld	GIMP	26	7.22	–	–
OSWorld	OS	24	6.67	–	–
OSWorld	LibreOffice Writer	23	6.39	–	–
OSWorld	VS Code	23	6.39	–	–
OSWorld	VLC	17	4.72	–	–
OSWorld	Thunderbird	15	4.17	–	–
macOSWorld	File management	29	20.28	29	0
macOSWorld	Media	12	8.39	0	12
macOSWorld	Productivity	35	24.48	16	19
macOSWorld	System and interface	29	20.28	29	0
macOSWorld	System apps	38	26.57	38	0

B Skill Source Statistics

All MMSkills are extracted from non-test trajectories. For OSWorld and macOSWorld, we use the Ubuntu and macOS subsets of OpenCUA trajectories as GUI skill sources (Wang et al., 2025a). For macOS, the raw OpenCUA trajectories do not directly follow the five macOSWorld categories; we therefore perform additional clustering and relevance filtering before assigning trajectories to the analysis categories below.

For VAB-Minecraft, we use the official training set as the source for extracting multimodal skill packages. For Super Mario Bros from LMGame-Bench, MMSkills are extracted from multiple runs over four source cases. In both settings, the skill-source data are disjoint from the final evaluation cases.

C Experiment Details

Across all evaluations, agents plan from visual environment observations rather than privileged state, using desktop screenshots for GUI tasks and game screenshots for game tasks. For OSWorld and macOSWorld, we run the full evaluations primarily on Amazon Web Services using the official benchmark images and task definitions. The agent interacts through the benchmark harness, and we use a maximum interaction budget of 20 steps for both GUI benchmarks. VAB-Minecraft and Super Mario Bros follow their official evaluation protocols.

Table 5 OpenCUA trajectory statistics used for GUI skill extraction. "Tasks" counts source trajectories, "Share" is the within-platform percentage, and "Clusters" is the number of Phase-0 semantic trajectory clusters used for downstream skill planning.

Platform	Domain	Tasks	Share	Clusters
Ubuntu	Chrome	718	17.1	17
Ubuntu	LibreOffice Impress	605	14.4	11
Ubuntu	VS Code	605	14.4	4
Ubuntu	OS	497	11.8	2
Ubuntu	GIMP	492	11.7	14
Ubuntu	LibreOffice Writer	490	11.7	3
Ubuntu	Thunderbird	300	7.1	11
Ubuntu	LibreOffice Calc	298	7.1	3
Ubuntu	VLC	200	4.8	8
macOS	Productivity	1,424	45.1	20
macOS	System apps	768	24.3	11
macOS	File management	341	10.8	9
macOS	Media	315	10.0	7
macOS	System and interface	309	9.8	12

Table 6 OSWorld MMSkill package statistics. "#Skills" counts unique skill packages, while "Skills/Task" reports the average number of skill matches assigned to evaluation tasks and therefore need not equal #Skills/#Tasks. Word statistics are median/mean over skill procedures. "Full/Focus" and "Before/After" report counts of those view types; "Transition Cards" counts state cards with at least one before/after transition view, with percentages over state cards. The Total/Avg row reports total counts and weighted averages; † marks a fitted value estimated from domain-level medians.

Domain	#Tasks	#Skills	Skills/Task	Words Med/Mean	#Cards	Cards/Skill	#Views	Views/Card	Full/Focus	Before/After	Transition Cards
Chrome	45	34	1.20	653 / 630.9	134	3.94	292	2.18	134/134	13/11	24 (17.9%)
GIMP	26	26	1.19	470 / 400.2	77	2.96	190	2.47	77/77	14/22	36 (46.8%)
Calc	47	26	1.36	278 / 278.1	79	3.04	184	2.33	79/79	7/19	26 (32.9%)
Impress	47	20	1.32	498 / 466.2	60	3.00	140	2.33	60/60	1/19	20 (33.3%)
Writer	23	23	1.13	264 / 289.2	71	3.09	144	2.03	71/71	1/1	2 (2.8%)
Multi-apps	93	20	1.19	574 / 502.0	82	4.10	164	2.00	82/82	0/0	0 (0.0%)
OS	24	37	1.21	544 / 539.8	139	3.76	283	2.04	139/139	5/0	5 (3.6%)
Thunderbird	15	25	1.20	508 / 542.5	87	3.48	192	2.21	87/84	6/15	21 (24.1%)
VLC	17	18	1.00	260 / 275.3	61	3.39	122	2.00	61/61	0/0	0 (0.0%)
VS Code	23	18	1.09	391 / 389.3	89	4.94	187	2.10	89/89	9/0	9 (10.1%)
Total / Avg.	**360**	**247**	**1.21**	**498.0† / 447.8**	**879**	**3.56**	**1898**	**2.16**	**879/876**	**56/87**	**143 (16.3%)**

For VAB-Minecraft, we use the official test set for evaluation. The training trajectories described in Appendix B are used only to generate reusable procedures, state cards, and keyframes; no test episodes are used during skill construction.

For Super Mario Bros from LMGame-Bench, we split the available game cases into disjoint source and evaluation subsets. The source cases are described in Appendix B, while a separate set of four held-out cases is used for final evaluation. This separation ensures that the generated skills capture reusable game situations rather than memorizing the measured episodes.

We evaluate both frontier and smaller multimodal models: Gemini 3.1 Pro, Gemini 3 Flash[1], Qwen3-VL-235B-A22B-Thinking (Bai et al., 2025), GLM-5V-Turbo (Team et al., 2026b), Kimi-K2.6 (Team et al., 2026a), and Qwen3-VL-8B-Instruct (Bai et al., 2025). For each base model, we compare *no-skill*, *text-only skill*, and *MMSkills* conditions. Unless otherwise stated, skill conditions use branch loading: text-only skills use the same branch mechanism without state cards or images, while MMSkills inspect selected state cards and multi-view keyframes before returning structured guidance to the main agent. Direct text-skill loading and direct multimodal loading are evaluated only as ablation

[1] https://storage.googleapis.com/deepmind-media/Model-Cards/Gemini-3-Flash-Model-Card.pdf

Table 7 macOSWorld MMSkill package statistics. "#Skills" counts unique skill packages, while "Skills/Task" reports the average number of skill matches assigned to evaluation tasks. Word statistics are median/mean over skill procedures. "Full/Focus" and "Before/After" report counts of those view types; "Transition Cards" counts state cards with at least one before/after transition view, with percentages over state cards.

Domain	#Tasks	#Skills	Skills/Task	Words Med/Mean	#Cards	Cards/Skill	#Views	Views/Card	Full/Focus	Before/After	Transition Cards
File management	29	30	1.03	358 / 374.5	62	2.07	128	2.06	62/62	4/0	4 (6.5%)
Media	12	25	2.08	378 / 400.8	55	2.20	116	2.11	55/55	6/0	6 (10.9%)
Productivity	35	59	1.69	324 / 330.2	125	2.12	261	2.09	125/125	11/0	11 (8.8%)
System/interface	29	88	3.03	282 / 285.5	182	2.07	380	2.09	182/182	16/0	16 (8.8%)
System apps	38	46	1.21	347 / 352.0	98	2.13	212	2.16	98/98	6/10	16 (16.3%)
Total / Avg.	**143**	**248**	**1.73**	**324 / 330.9**	**522**	**2.10**	**1097**	**2.10**	**522/522**	**43/10**	**53 (10.2%)**

Table 8 Game benchmark MMSkill package statistics. Word statistics are median/mean over skill procedures and plans. "Full/Focus" and "Before/After" report counts of those view types; "Transition Cards" counts state cards with at least one before/after transition view, with percentages over state cards. † marks a fitted value estimated from the available before/after view counts.

Benchmark	#Skills	Skill Words Med/Mean	Plan Words Med/Mean	#Cards	Cards/Skill	#Views	Views/Card	Full/Focus	Before/After	Transition Cards
VAB-Minecraft	24	278.5 / 281.7	68.0 / 68.4	79	3.29	185	2.34	79/79	8/19	20 (25.3%)
Super Mario Bros	10	374.0 / 370.8	280.0 / 271.0	34	3.40	48†	1.41†	34/0	5/9	14 (41.2%)†

variants.

D Branch-Loaded Runtime Algorithm

Algorithm 1 summarizes the branch-loaded runtime loop. Candidate skills are selected before task execution, while branch calls occur only when the main agent decides to consult a specific skill. The main trajectory receives the structured guidance G_t rather than the full multimodal skill package.

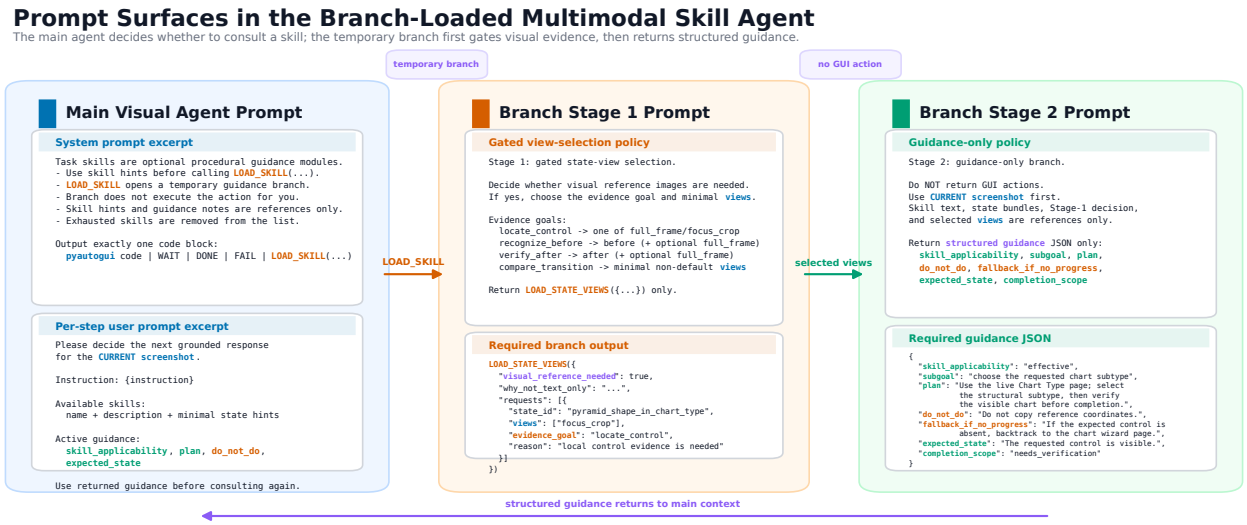

Figure 5 Prompt surfaces used by the branch-loaded multimodal skill agent. The main agent prompt decides whether to act directly or consult a skill branch, Stage 1 selects the relevant state cards and keyframe views, and Stage 2 returns compact structured guidance to the main agent.

E MMSkillAgent Prompt Templates

This section reports the prompt templates used by the branch-loaded MMSkillAgent. Dynamic fields are shown as placeholders such as `{instruction}`, `{available_skills}`, and `{previous_steps}`. The implementation instantiates these templates with the current screenshot, recent trajectory, execution feedback, candidate skills, state-card

Algorithm 1 Branch-loaded MMSkill Agent

Require: Skill library $\mathcal{M}$, task instruction I, visual environment Env

Initialize history $H_0 \leftarrow \emptyset$
Pre-recall candidate skills $\mathcal{C}_I \leftarrow \text{PreRecall}(I, \mathcal{M})$
for $t = 1, 2, \ldots$ **do**
  Observe current visual observation O_t from Env
  Main agent chooses either action A_t or skill request $M_t \in \mathcal{C}_I$
  if the main agent chooses action A_t **then**
    Execute A_t in Env and update H_t
  else
    Unpack $M_t = (D_t, P_t, S_t, K_t)$
    Stage 1: $(J_t, R_t) \leftarrow \text{SelectViews}(O_t, H_{t-1}, P_t, S_t)$
    Load $V_t \leftarrow \{K_j^v : j \in J_t,\ v \in R_{t,j}\}$
    Stage 2: $G_t \leftarrow \text{PlanBranch}(O_t, H_{t-1}, P_t, \{S_j : j \in J_t\}, V_t)$
    Choose grounded action $A_t \leftarrow \pi_{\text{main}}(O_t, H_{t-1}, G_t)$
    Execute A_t in Env and update H_t
  end if
  if the task is verified complete **then**
    return success
  end if
end for

summaries, and selected keyframe views. The Stage-2 JSON contains a few implementation-facing fields beyond Eq. 2; they are collapsed into G_t in the method description.

Main-Agent Skill-Calling System Prompt

Role. Follow the user instruction to perform desktop computer tasks. You control the computer using Python code with `pyautogui`. At each step, you receive the current screenshot and recent visible trajectory history. Use the current screenshot to decide the next action; do not assume previous clicks succeeded.

Skill consultation policy.

- Task skills are optional procedural planners only.
- The final user message includes each non-exhausted skill's short description and minimal runtime state hints. Use these hints to judge whether a skill is genuinely relevant before calling `LOAD_SKILL(...)`.
- Call `LOAD_SKILL("<exact_skill_name>")` only when the current screenshot, recent steps, and skill hints suggest that extra procedural guidance is useful.
- `LOAD_SKILL(...)` opens a temporary planner branch for extra skill-guided reasoning; it does not execute the action.
- Skill hints and planner notes are references only, never coordinate templates.
- Each skill may be consulted at most `{consult_limit}` times in one trajectory. Exhausted skills are removed from the available-skill list and must not be called again.

Available skills. `{available_skills}` lists non-exhausted candidate skills for the task.

Action rules.

- Use `pyautogui` only for GUI actions. Do not use `pyautogui.locateCenterOnScreen` or `pyautogui.screenshot()`.
- Each response must be self-contained and must not rely on variables from previous steps.
- If a click does not work, revise the target from the new screenshot instead of repeating the same guess.
- Prefer short, direct, grounded actions over long speculative scripts; avoid repetitive unproductive loops.
- Before outputting `DONE`, verify that the full user instruction has been completed, not only a local subgoal.

Output interface. Return exactly one code block containing one of: Python code using `pyautogui`, `WAIT`, `DONE`, `FAIL`, or `LOAD_SKILL("<exact_skill_name>")`. Do not mix Python code with a skill call, do not load more than one skill, and do not return prose outside the code block. If returning Python, include concise # comments. Use `WAIT` only for

loading UI, DONE only after full verification, and FAIL only when the task is truly impossible. Canonical outputs include LOAD_SKILL("Example_Skill_Name") and a single grounded action such as pyautogui.click(120, 54).

Coordinate and task context. Use the declared screen resolution for all pyautogui coordinates. The computer password is available as {client_password} when needed. The task is {instruction}.

Main-Agent Per-Step User Instruction

Decision request. Decide the next grounded response for the current screenshot. Return either the next GUI action or LOAD_SKILL(...) when extra procedural guidance is useful.

Per-step context.

- **Instruction:** {instruction}
- **Available non-exhausted skills:** {skills_with_state_previews}, including each skill name, short description, and minimal when-to-use state hints.
- **Active planner memo:** {active_memo}
- **Planner notes returned in this step:** {current_step_planner_summaries}
- **Previous steps:** {previous_steps}, including full model responses and action comments.
- **Execution feedback:** optional feedback for the current step and optional loop-warning diagnostics.
- **Screen resolution:** {screen_resolution_prompt}.

Grounding rules.

- Ground every action in the current screenshot.
- Planner notes are fallible references; re-decide the real action from the current screenshot, recent history, and execution feedback.
- Treat state hints, selected reference views, and planner notes as references only, never coordinate templates.
- If no listed skill is clearly useful, act directly from the current screenshot.
- If planner notes already exist for this step, use them before consulting another branch.
- If recent actions repeat without progress, change strategy.
- Before DONE, verify the full instruction; if returning Python, include concise comments.

Branch Stage 1 Prompt: Gated State-View Selection

Branch reference package. The branch receives the requested call LOAD_SKILL("{skill_name}"), the selected skill text, runtime state bundles, and compact state-card manifests. These materials are supplemental procedural references only. Stage 1 must decide whether visual reference images are needed at all and, if so, which state IDs and view types should be loaded. The main agent, not the branch, will choose the concrete GUI action.

Role. You are inside Stage 1 of a temporary state-view selection branch for a single desktop step. Decide whether visual reference images are needed before planner reasoning and which evidence goal they should serve.

View semantics.

- full_frame: global placement and window context.
- focus_crop: detailed control localization.
- before: pre-change state, useful for recognizing whether the UI is still before a change and for avoiding repeated toggles.
- after: target completion state, useful for verifying the result after save, enable, format, or apply operations.

Evidence goals.

- locate_control: request exactly one of full_frame or focus_crop.
- recognize_before: request before, optionally with full_frame.
- verify_after: request after, optionally with full_frame.

- `compare_transition`: request minimal transition evidence; avoid defaulting to the `full_frame+focus_crop` pair and prefer `before/after` when useful.

Visual gating policy. First decide `visual_reference_needed`. If the useful help is a generic shortcut, formula, file operation, stable menu path, or textual procedure, default to `false`. Load images only for state transitions, visual result verification, or complex UI-state recognition where text alone is likely insufficient. Keep the request minimal: at most `{max_states}` states and `{max_views}` total views.

Input fields. Stage 1 receives `{instruction}`, `{previous_steps}`, environment feedback from the previous step, loop warnings if present, the screen-resolution prompt, and the current screenshot.

Output interface. Return exactly one code block containing one `LOAD_STATE_VIEWS(...)` call. Its JSON payload contains:

- `"visual_reference_needed"`: true or false;
- `"why_not_text_only"`: why text-only is insufficient, or why no images are needed;
- `"requests"`: a list of objects, each with exact `"state_id"`, exact `"views"`, `"evidence_goal"`, and `"reason"`.

When `"visual_reference_needed"` is false, `"requests"` must be empty. Do not return Python code, planner JSON, `WAIT`, `DONE`, `FAIL`, `LOAD_SKILL`, or prose outside the code block.

Canonical examples. A transition request sets `"visual_reference_needed": true` and requests a state with `"views": ["before", "after"]` under `"evidence_goal": "compare_transition"`. A text-only branch sets `"visual_reference_needed": false`, gives a brief reason in `"why_not_text_only"`, and returns `"requests": []`.

Branch Stage 2 Prompt: Planner JSON

Selected evidence package. Stage 2 receives the Stage-1 selection record, including the evidence goal, selected states, requested view types, reasons, when-to-use conditions, verification cues, and any loaded keyframe views. Loaded views are supplemental references only and are never coordinate templates.

Role. You are inside Stage 2 of a temporary planner-only skill consultation branch for a single desktop step. Do not return a GUI action. Return a structured planner summary for the current state.

Branch rules.

- Do not return Python code, `WAIT`, `DONE`, `FAIL`, `LOAD_SKILL`, `LOAD_SKILL_IMAGE`, or `LOAD_STATE_VIEWS`. Do not request another skill in this branch.
- Use the current screenshot first. Skill text, runtime state bundles, Stage-1 decisions, and loaded reference views are supplemental only.
- If Stage 1 chose no visual references, respect that decision and avoid inventing image-based assumptions.
- If the skill is ineffective for the current state, say so clearly and avoid forcing the plan toward it.
- Treat reference views as state references, never as coordinate templates.

Planning requirements.

- `subgoal`: next immediate local milestone under the live UI.
- `plan`: longer-range route grounded in the current screenshot, including the relevant UI surface, the next 2–4 actions/checks/transitions, and the cue that means advance versus re-plan.
- `do_not_do`: the likely wrong path or skill-induced mistake to avoid.
- `fallback_if_no_progress`: a concrete alternate route if the skill-guided path stalls.
- `expected_state`: visible screenshot cues the main agent should aim to reveal next.
- `completion_scope`: whether the branch only advances a local step, still needs verification, or may be complete after verification.

Per-step input fields. Stage 2 receives `{instruction}`, `{stage1_decision}`, `{selected_state_views}`, `{previous_steps}`, environment feedback, optional loop warnings, the screen-resolution prompt, and the live screenshot, which is more authoritative than any skill reference view.

Output interface. Return exactly one code block containing one JSON object with keys: `"skill_applicability"`, `"subgoal"`, `"plan"`, `"do_not_do"`, `"fallback_if_no_progress"`, `"expected_state"`, and `"completion_scope"`. The

values of "skill_applicability" are "effective", "ineffective", or "uncertain"; the values of "completion_scope" are "local_only", "needs_verification", or "maybe_complete". Do not return prose outside the code block.

Canonical example shape. A valid planner object may mark the skill as "effective", set a local "subgoal" such as opening the visible settings surface, give a grounded multi-step "plan", block a likely repeated or irrelevant click through "do_not_do", provide a concrete fallback route, and describe the next visible "expected_state" with "completion_scope": "local_only".

F Additional Behavioral Shift Analysis

Figure 6 complements Figure 4 with the same OSWorld behavioral analysis for GLM-5V and Kimi-K2.6.

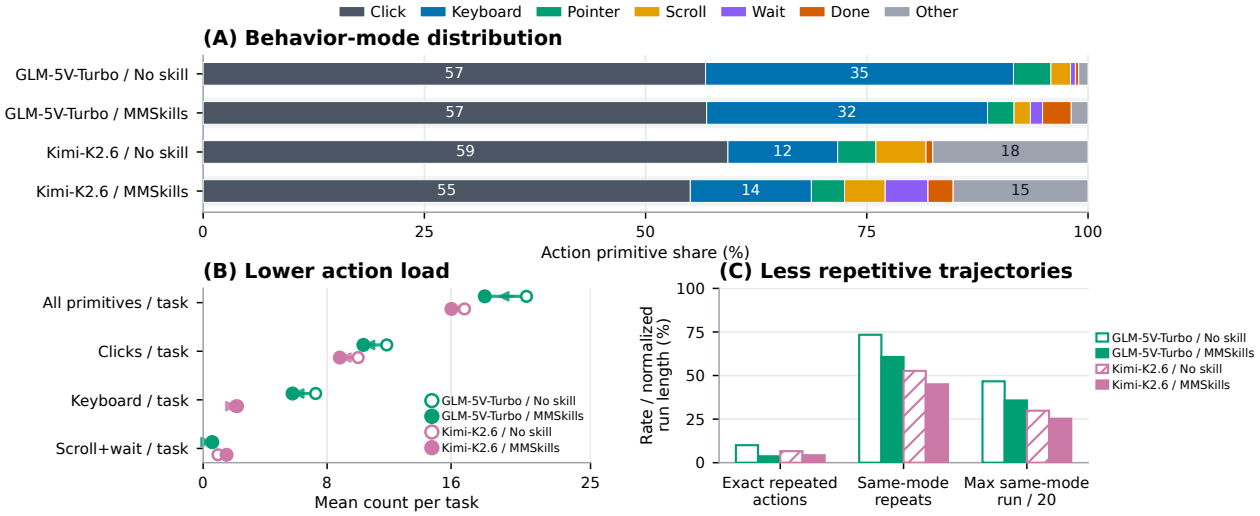

Figure 6 Behavioral shifts induced by MMSkills on OSWorld for GLM-5V and Kimi-K2.6. The panels follow the same metrics as Figure 4: action primitive distribution, low-level primitives per task, and repetitive behavior statistics.

G Interaction Case Studies

Figures 7 and 8 show two representative OSWorld interaction traces. The first case illustrates a LibreOffice Calc workflow in which the agent consults different spreadsheet skills at different stages of table construction. The second case illustrates a terminal file-organization task where branch guidance helps move past an initially brittle command and then verifies the final archive structure.

H Broader Impact

MMSkills are intended to make visual agents more reliable by externalizing reusable multimodal procedural knowledge. Potential benefits include improved desktop automation, reduced repeated trial-and-error interactions, better support for smaller models, and more reusable agent knowledge across GUI and game-like visual environments. At the same time, more capable visual agents may also increase the risk of unwanted automation, misuse in interactive software, or accidental actions in sensitive environments. Multimodal skill packages can also contain screenshots or cropped visual evidence, so their construction should avoid private or proprietary user data unless appropriate consent, filtering, and access controls are in place. In this work, we construct skills from public non-evaluation trajectories and store compact state evidence rather than raw demonstrations whenever possible. Future deployments should combine MMSkills with permission controls, task-level safety policies, sensitive-information filtering, and auditing of generated skill packages before they are made available to autonomous agents.

Case 1: LibreOffice Calc Table Construction

Two different Calc skills are consulted; visual evidence is loaded only for the worksheet-tab operation.

Task "Create a table with two column headers (Month, Total) in a new sheet named Sheet2 to show total sales for all months."

score 1.0 | 7 actions | 2 skills

GUI action | LOAD_SKILL | Branch guidance | Evidence gate

Turn 0	LOAD	
Input		Current sheet is Sheet1. The task requires a new destination sheet named Sheet2 before any table content is entered.
Output		LOAD_SKILL("Manage Calc Worksheets and Cross Sheet Data")
Effect		A temporary skill branch is opened before taking a GUI action.
Turn 0B	Branch	
Input		Skill package + live screenshot. Stage 1 requests focus crops for the sheet-tab plus button and rename-sheet field.
Output		structured guidance: create and activate Sheet2 first; do not type Month/Total on Sheet1.
Effect		Guidance returns to the main agent in the same interaction step.
Turn 1	Action	
Input		Guidance says to create Sheet2. Bottom tab strip is visible with the small plus control.
Output		pyautogui.click(103, 965)
Effect		New Sheet2 tab is created and becomes active.
Turn 2	Action	
Input		Sheet2 is active; cell A1 is ready for the first header.
Output		typewrite("Month"); move to B1; typewrite("Total")
Effect		Header row on Sheet2 becomes Month | Total.
Turn 3	Action	
Input		Sheet2 has headers. Active cell is ready under the Month column.
Output		typewrite("Jan"..."Jun") into A2:A7
Effect		Month labels Jan through Jun are entered.
Turn 3B	LOAD	
Input		The table skeleton is ready, but totals must be formula-based and reference Sheet1.
Output		LOAD_SKILL("Use Formulas and Functions in Calc Cells")
Effect		A second, different skill branch is opened.
Turn 3C	Branch	
Input		Skill package + live screenshot. Stage 1 decides no reference image is needed for this stable formula-entry step.
Output		structured guidance: move to B2; enter formulas referencing Sheet1; verify the first visible result.
Effect		The main agent receives formula-entry guidance without loading extra images.
Turn 4	Action	
Input		Sheet2 shows Jan-Jun in A2:A7. Active cell is below the month list.
Output		move to B2; enter =SUM(Sheet1.B2:B11) ... =SUM(Sheet1.G2:G11)
Effect		Total column B2:B7 is filled with formula results.
Turn 5	Action	
Input		Sheet2 table is filled. The agent checks whether the source sheet contains additional months.
Output		click Sheet1 tab
Effect		Source sheet is inspected before finalizing.
Turn 6	Action	
Input		After checking Sheet1, the final artifact must be verified on Sheet2.
Output		click Sheet2 tab
Effect		The completed result sheet is visible again.
Turn 7	Done	
Input		Sheet2 displays Month/Total and formula-derived totals.
Output		click B2 to verify formula bar; DONE
Effect		Formula bar shows a Sheet1 reference. The task is marked complete with score 1.0.

Figure 7 Representative interaction case with branch-loaded MMSkills: LibreOffice Calc table construction. Colored turn labels distinguish direct GUI actions, skill loading, branch guidance, evidence-gated reasoning, and final completion.

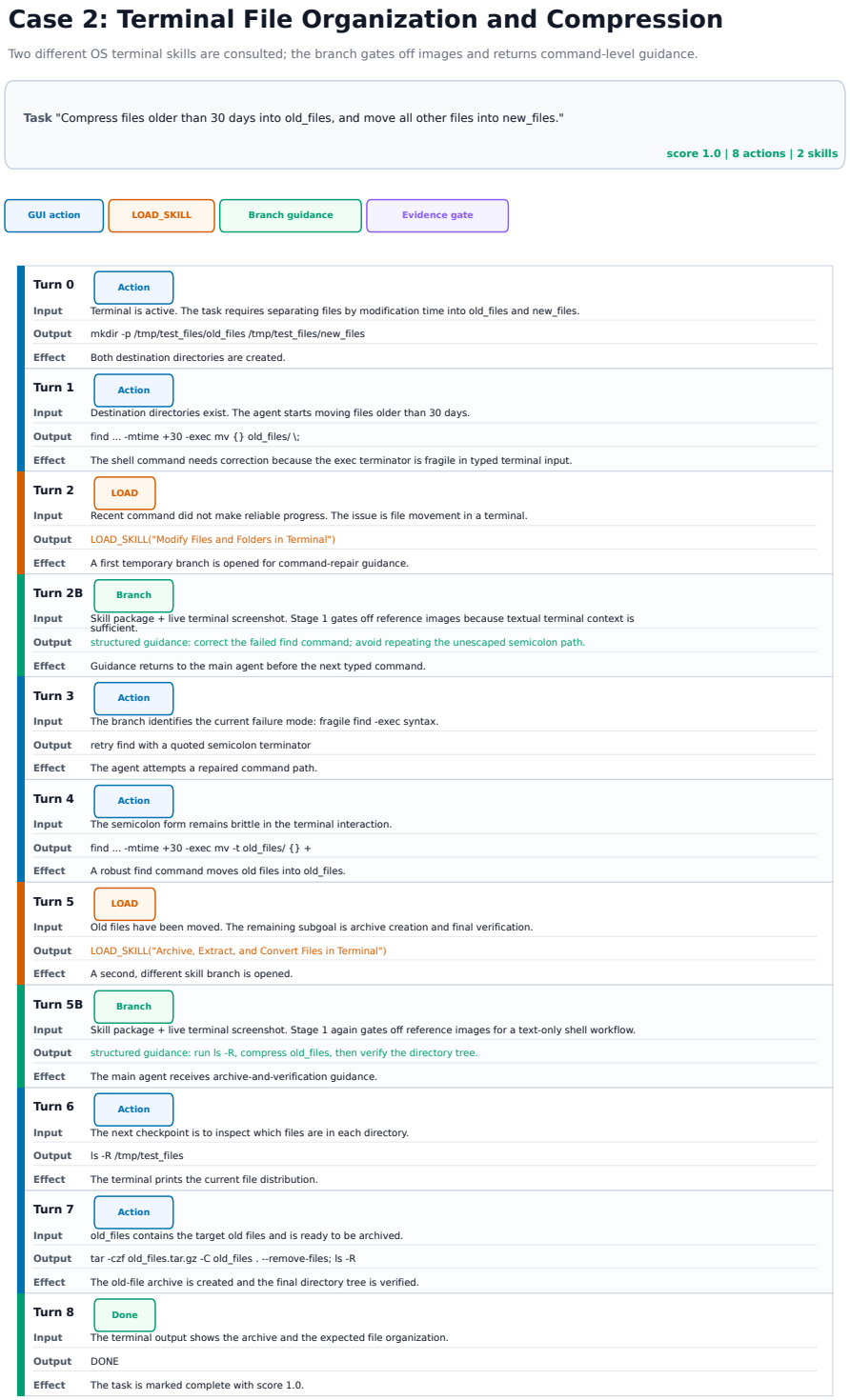

Figure 8 Representative interaction case with branch-loaded MMSkills: terminal file organization and compression. Colored turn labels distinguish direct GUI actions, skill loading, branch guidance, evidence-gated reasoning, and final completion.

I Use of LLMs

Large language models are used in this work as both research artifacts and research assistants. Methodologically, LLM-based agents are used in the skill-generation pipeline to process and filter trajectories, propose reusable procedures, draft state cards, and generate multimodal skill packages under human-designed schemas and quality checks. LLMs also serve as the evaluated visual agents in the benchmark results. In addition, LLM tools were used during manuscript preparation for editing, polishing, and organizing written content. The authors remained responsible for experimental design, result interpretation, citation checking, and final paper content.

J Detailed Related Work

This section provides the expanded related-work discussion summarized in Section 4.

Skills for agents. Skill reuse has a long history in temporal abstraction for reinforcement learning and motor primitives for robotics (Sutton et al., 1999; Ijspeert et al., 2013). Recent LLM agents have made skills a practical interface for storing and composing procedural knowledge in language-conditioned environments. Early systems connected language models to action by grounding language in affordances (Ahn et al., 2022), emitting executable programs (Liang et al., 2023), or interleaving reasoning and acting (Yao et al., 2023); adjacent code- and tool-agent work studies robust tool-call data loops, search-based code refinement, and adversarial test-case generation (Zhang et al., 2025; Li et al., 2025b, 2026a). Reflection mechanisms then made agent behavior more persistent across attempts (Shinn et al., 2023). In open-ended environments, systems such as DEPS, Voyager, and JARVIS-1 showed that large models can use language, stored experience, and self-generated programs to acquire or reuse behaviors over extended task horizons (Wang et al., 2024, 2023a,b). These works motivate our focus on procedural reuse, but their reusable knowledge is primarily textual, symbolic, or programmatic.

More recent work treats skills as an explicit substrate for agent improvement. SkillWeaver distills web exploration into reusable API-like skills (Zheng et al., 2025); CUA-Skill builds a parameterized skill base with execution and composition graphs for computer-using agents (Chen et al., 2026); SkillX automatically constructs hierarchical skill knowledge bases from agent experience (Wang et al., 2026a); EvoSkill studies automated skill discovery through failure analysis in multi-agent settings (Alzubi et al., 2026), where decentralized coordination and scalable improvement are also central concerns (Yang et al., 2025c; Shao et al., 2026a); SkillClaw evolves shared skills from multi-user trajectories (Ma et al., 2026); and SkillRL co-evolves a hierarchical skill library with reinforcement learning (Xia et al., 2026). A recent survey frames agent skills as portable packages of instructions, code, and resources loaded through progressive disclosure (Xu and Yan, 2026). A complementary perspective treats accumulated agent experience as long-term memory: Generative Agents maintain a memory stream that supports recall, reflection, and planning (Park et al., 2023), while MemGPT introduces an OS-style memory hierarchy that pages information in and out of the model's working context (Packer et al., 2024). MMSkills follows this broader move toward modular procedural knowledge, but changes the unit being stored: instead of treating skills mainly as text, code, APIs, or execution graphs, we define a skill package whose central evidence is a set of visually grounded runtime states. Branch loading also takes inspiration from memory-paging ideas, by inspecting selected multimodal evidence in a temporary branch rather than flooding the main context.

This emerging ecosystem has also motivated dedicated evaluation of skill utility. SkillsBench measures how skills affect agent performance across diverse tasks (Li et al., 2026b), SkillTester evaluates utility and security risks of agent skills (Wang et al., 2026b), and recent work studies skill usage under more realistic retrieval and adaptation settings (Liu et al., 2026). These benchmarks show that skills are not automatically beneficial; their value depends on relevance, compactness, selection, and safe use, especially as self-evolving agents may introduce emergent risks (Shao et al., 2026b). Our work addresses a complementary question for visual agents: what evidence should a skill expose, and how should that evidence be loaded, when correct use depends on the current visual state?

The closest line to our work is multimodal and GUI-specific skill augmentation. Mirage-1 introduces hierarchical multimodal skills for GUI agents and uses them with search to support long-horizon control (Xie et al., 2025); XSkill continually extracts experiences and skills for multimodal agents from visually grounded rollouts (Jiang et al., 2026); MuSEAgent studies stateful experiences for multimodal reasoning agents (Wang et al., 2026c); and CUA-Skill builds computer-use skills as parameterized procedures and execution graphs (Chen et al., 2026). MMSkills differs in emphasis: we define the skill artifact around reusable visual state evidence, not only around executable procedure structure or memory accumulation. Each skill is organized around when-to-use conditions, visible cues, verification cues, and multi-view state evidence, and the runtime first selects the relevant evidence before exposing it to the main agent. This makes the contribution a representation and loading mechanism for multimodal procedural cues, rather than another text skill library or GUI action graph.

Visual agents. Visual agents have rapidly advanced from web navigation to general computer use. Benchmarks such as Mind2Web and WebArena established realistic web-agent evaluation beyond synthetic interfaces (Deng et al., 2023; Zhou et al., 2024); VisualWebArena showed that many web tasks require visual grounding rather than text-only

reasoning (Koh et al., 2024); and WebVoyager demonstrated end-to-end web interaction with large multimodal models on real websites (He et al., 2024). The same trend appears in mobile, desktop, and embodied settings: Android in the Wild and AndroidWorld study device control from visual UI observations (Rawles et al., 2023, 2025), OSWorld and macOSWorld evaluate agents in real operating-system environments (Xie et al., 2024; Yang et al., 2025b), RiOS-World evaluates risks in multimodal computer-use agents (Yang et al., 2025a), and VisualAgentBench includes VAB-Minecraft and VAB-OmniGibson for open-world and household embodied interaction (Liu et al., 2024a).

Model and framework work has likewise moved toward visually grounded action, reflecting the shared multimodal objective of aligning visual and textual representations (Liu et al., 2024b; Zhang et al., 2024). SeeClick trains GUI grounding for screenshot-only agents (Cheng et al., 2024); CogAgent introduces a visual language model dedicated to GUI understanding and operation (Hong et al., 2024); OS-ATLAS learns a foundation action model for GUI control (Wu et al., 2024); UI-TARS develops native GUI agents that perceive screenshots and emit keyboard/mouse actions (Qin et al., 2025); SeeAct builds web agents around general-purpose vision-language models (Zheng et al., 2024); AppAgent learns smartphone skills from on-device demonstrations (Zhang et al., 2023); OmniParser provides a pure-vision parser that turns screenshots into structured GUI elements (Lu et al., 2024); and Agent S provides a general computer-use framework built around GUI interaction (Agashe et al., 2024). These systems improve the agent's perceptual and action interface. MMSkills instead targets the external knowledge layer used by such agents. A stronger GUI action model may click more accurately, but it still benefits from knowing which procedural state matters, which visual cue confirms progress, and which state indicates that a skill should not be applied. MMSkills represents that knowledge as a compact, reusable multimodal skill package.

GUI grounding benchmarks. Alongside task-completion benchmarks, a separate line of work measures how reliably GUI agents can localize UI elements from natural-language instructions. ScreenSpot-Pro extends earlier ScreenSpot evaluations to high-resolution, professional desktop environments, where target elements often occupy less than 0.1% of the screen and the strongest grounding models still fall well below human performance (Li et al., 2025a). Gou et al. (2025) push toward universal visual grounding that lets agents identify GUI elements purely from screenshots, in the spirit of how humans navigate digital interfaces. MMBench-GUI organizes evaluation hierarchically, from content understanding and element grounding to task automation and multi-agent collaboration (Wang et al., 2025b), and DeskVision contributes a large-scale desktop dataset and evaluation suite that broadens grounding research across operating systems (Xu et al., 2025). These benchmarks isolate the perceptual layer of visual agents. MMSkills is complementary: rather than improving where to click, it provides procedural and visual evidence about which state matters at each step, and lets the underlying grounding capability translate that evidence into precise actions.

Long-context reliability. Recent studies have shown that simply enlarging the context window does not guarantee that all evidence is used effectively. Liu et al. (2023) report that language models often fail to retrieve information placed in the middle of long contexts, and benchmarks such as LongBench reveal substantial degradation as the input grows in length and modality (Bai et al., 2024). These observations motivate our branch-loaded design: rather than directly inserting state cards, multi-view keyframes, and transition examples into the main agent context, the runtime first inspects selected evidence in a temporary branch and returns a compact structured guidance tuple. This isolates expensive multimodal evidence reading from action generation, and avoids the long-context failure modes that arise when reference views and live observations compete for the same context window.